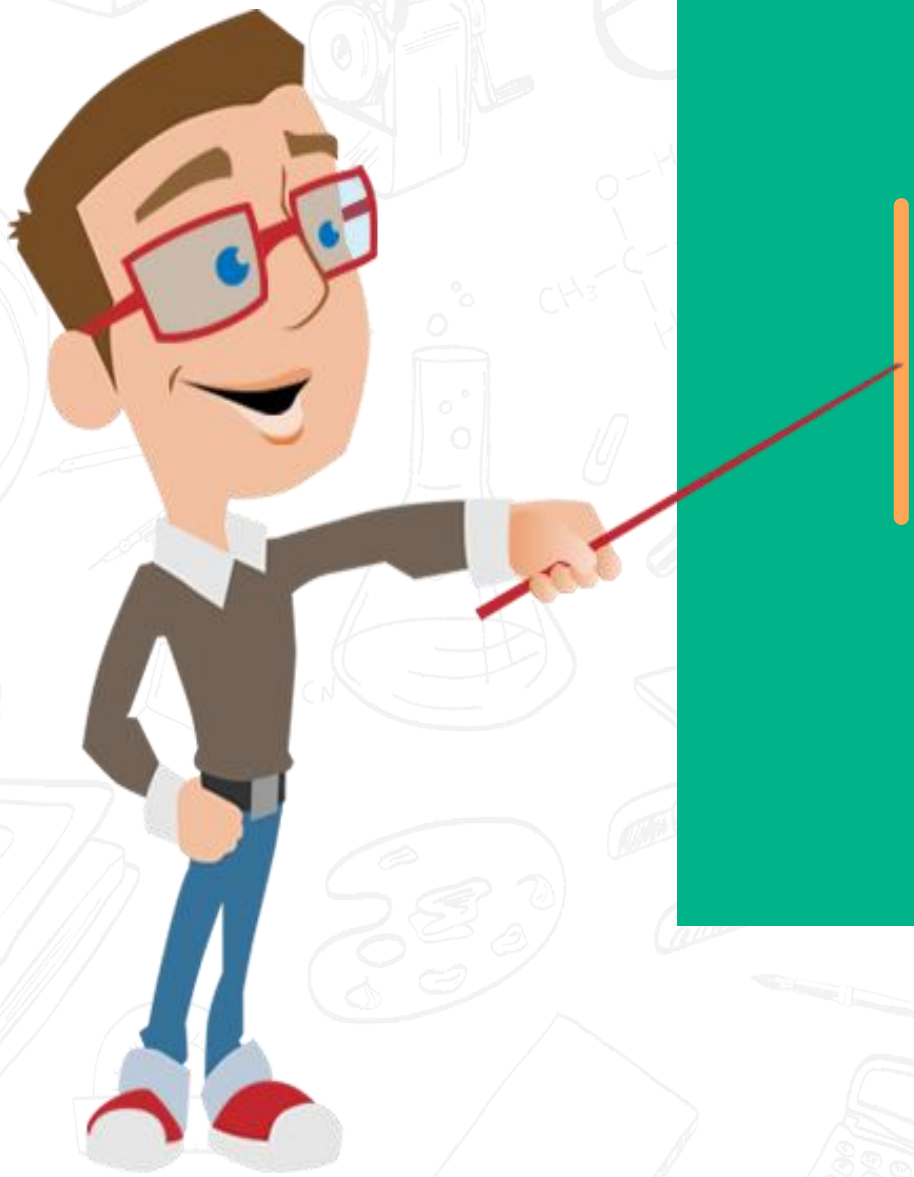


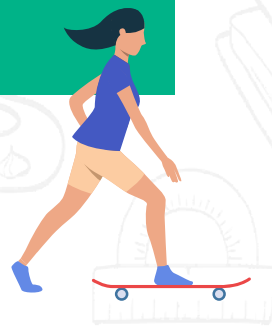
# 机器人操作系统（双语）

## Robot Operating System (Bilingual)

WHERE THERE'S A WILL, THERE'S A WAY

授课教师：郄(qiè)龙飞

机器人工程



投票 最多可选1项

设置

**请同学们评价下 ROS 单元 的授课效果 ☺ 🎉：**

A **非常满意，授课效果出色，完全满足期望**

B **满意，对授课效果感到满意，但还有改进的空间**

C **一般，授课效果一般，有一些改进的必要性**

D **不满意，对授课效果不太满意，有很多改进的空间**

提交

主观题 10分

设置

**对授课方式的一些建议和意见，可以从授课风格、讲义和教学内容、互动和参与、课程组织和时间管理、额外的意见等方面谈谈你的想法**

作答

# Overview lesson 11-1

1 rqt tools

2 ROS APIs

3

4

## Objectives

1、了解 rqt 常用插件的使用方法

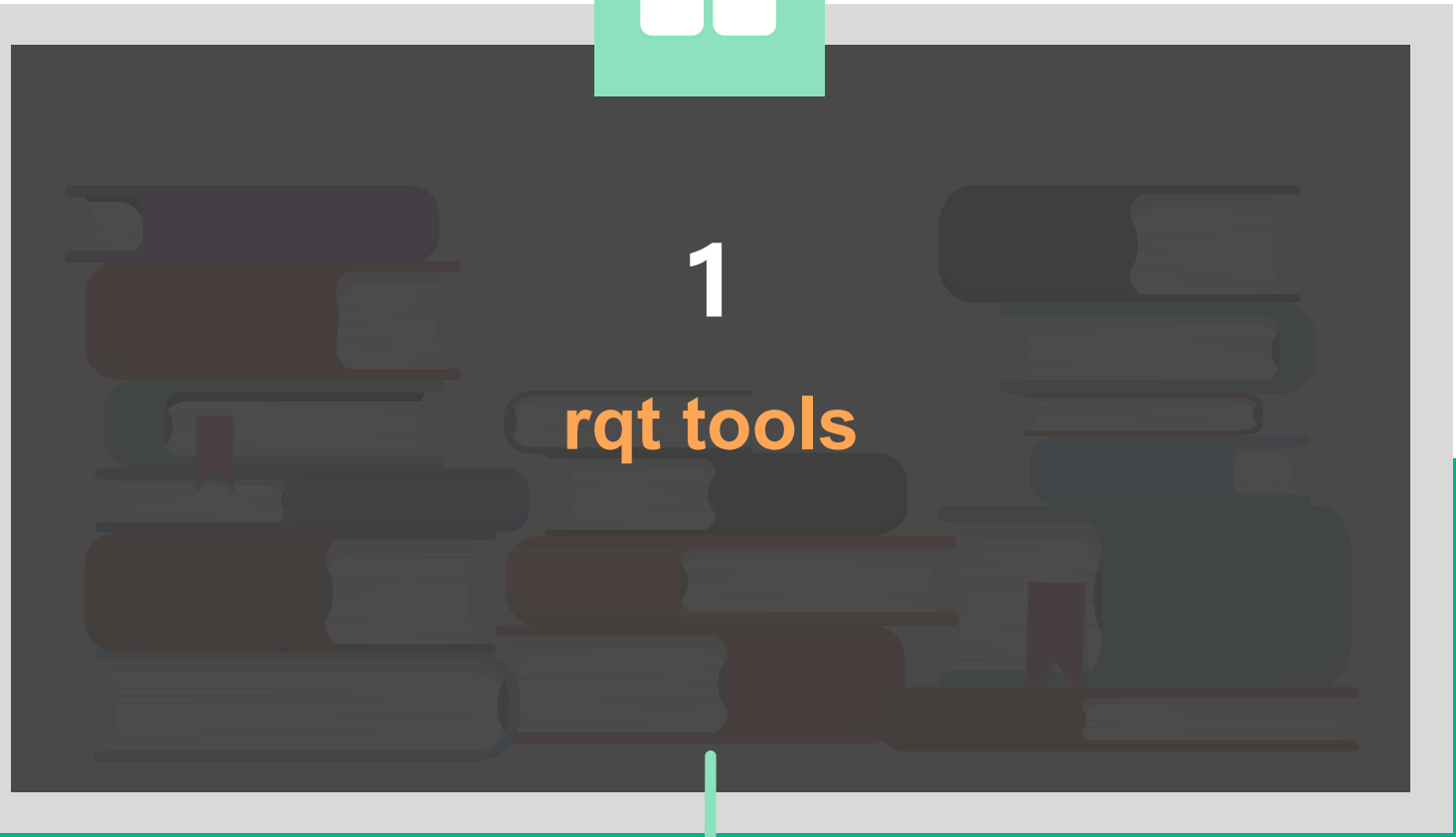
2、了解 ROS 中的 API 函数



# Pre-assessment

1、是否用过rqt工具?

2、是否了解API含义?



# 1

# rqt tools

# 1.1 rqt

## 1.1.1 Overview

- 在 ROS 中，rqt 工具箱可以以图形化操作代替命令行操作，操作过程更加直观和便利，有效提高操作效率，提升用户体验

- **概念**
  - rqt 是 ROS 基于 QT 框架，针对机器人开发的一系列可视化工具集合

- **作用**
  - rqt 可以方便的实现 ROS 可视化调试，并且在同一窗口中打开多个部件，提高开发效率，优化用户体验

- **组成**
  - rqt 工具箱组成有三大部分：
    1. rqt——核心实现，开发人员无需关注
    2. rqt_common_plugins——rqt 中常用的工具套件
    3. rqt_robot_plugins——运行中和机器人交互的插件(比如：rviz)

# 1.1.2 rqt installation and startups

## 安装

- 若安装 ROS 时选择 desktop-full 版本，则自带 rqt 工具箱
- 否则，需要通过以下方式安装：

```
$ sudo apt install ros-noetic-rqt
```

```
$ sudo apt install ros-noetic-rqt-common-plugins
```

## 启动

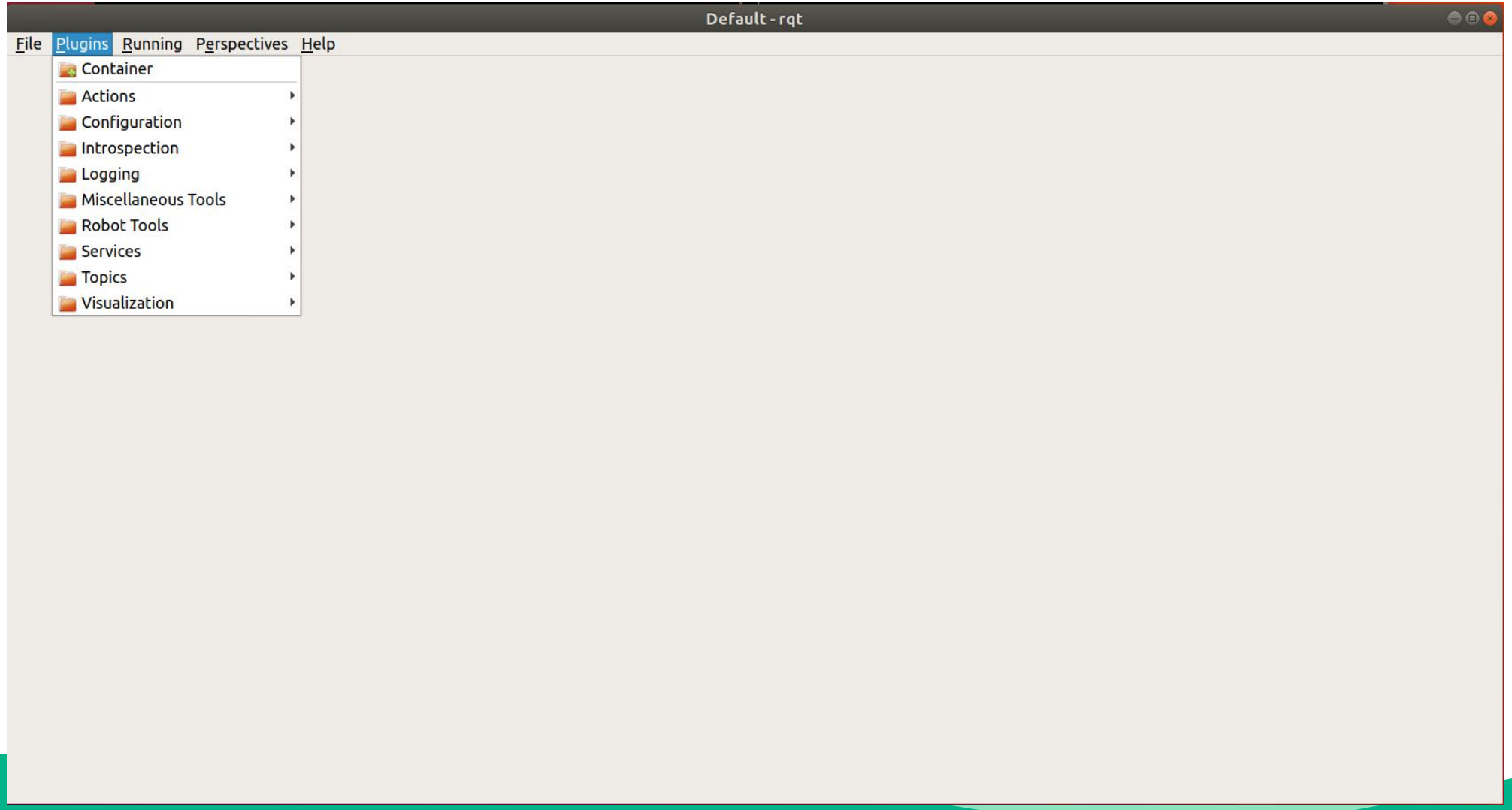
- rqt的启动方式有两种：启动roscore之后
  1. 方式1：rqt
  2. 方式2：rosrun rqt_gui rqt_gui

## 基本使用

- 启动 rqt 之后，可以通过 plugins 添加所需的插件

# 1.1 rqt

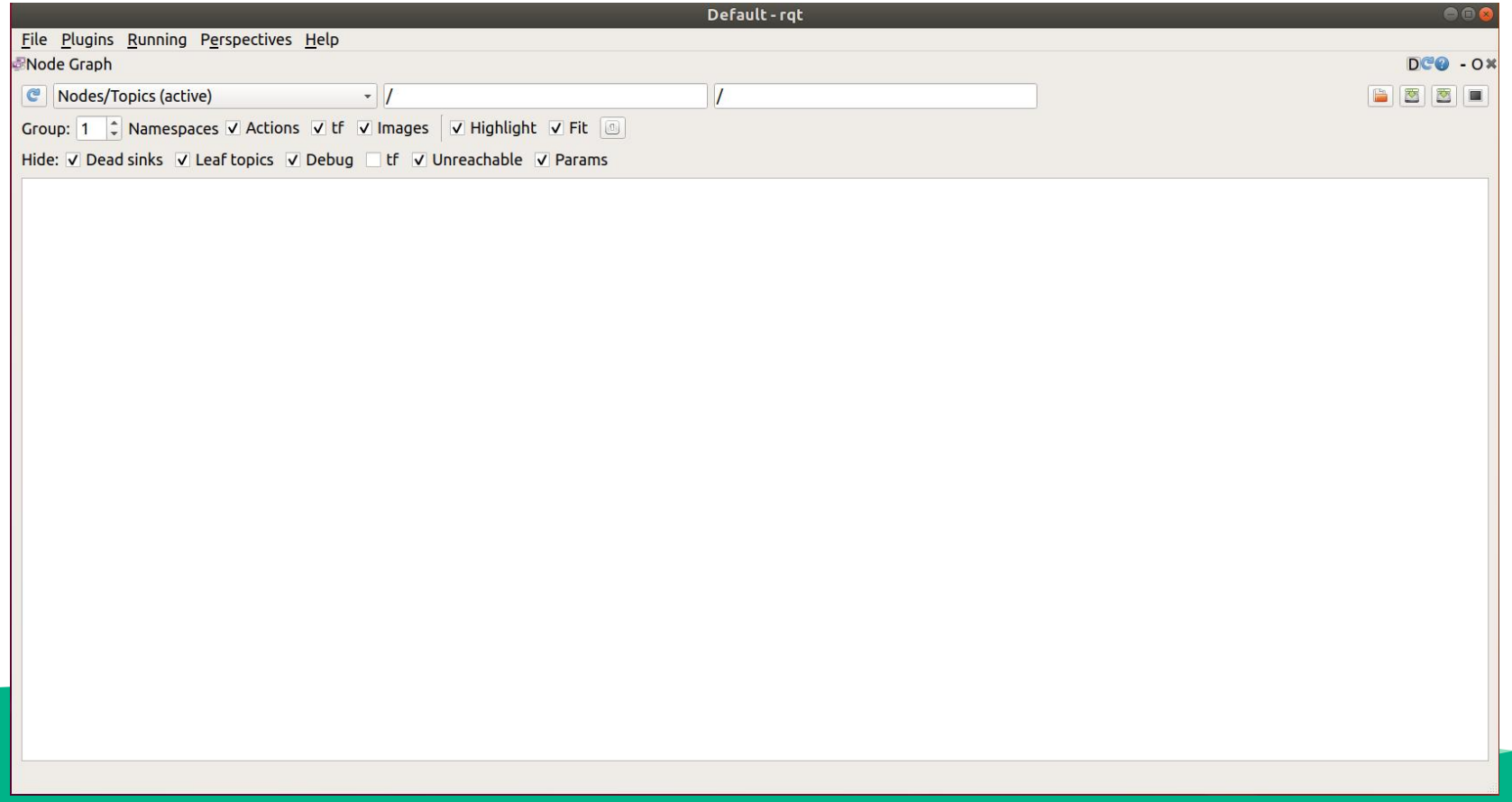
## 1.1.2 rqt installation and startups



# 1.1 rqt

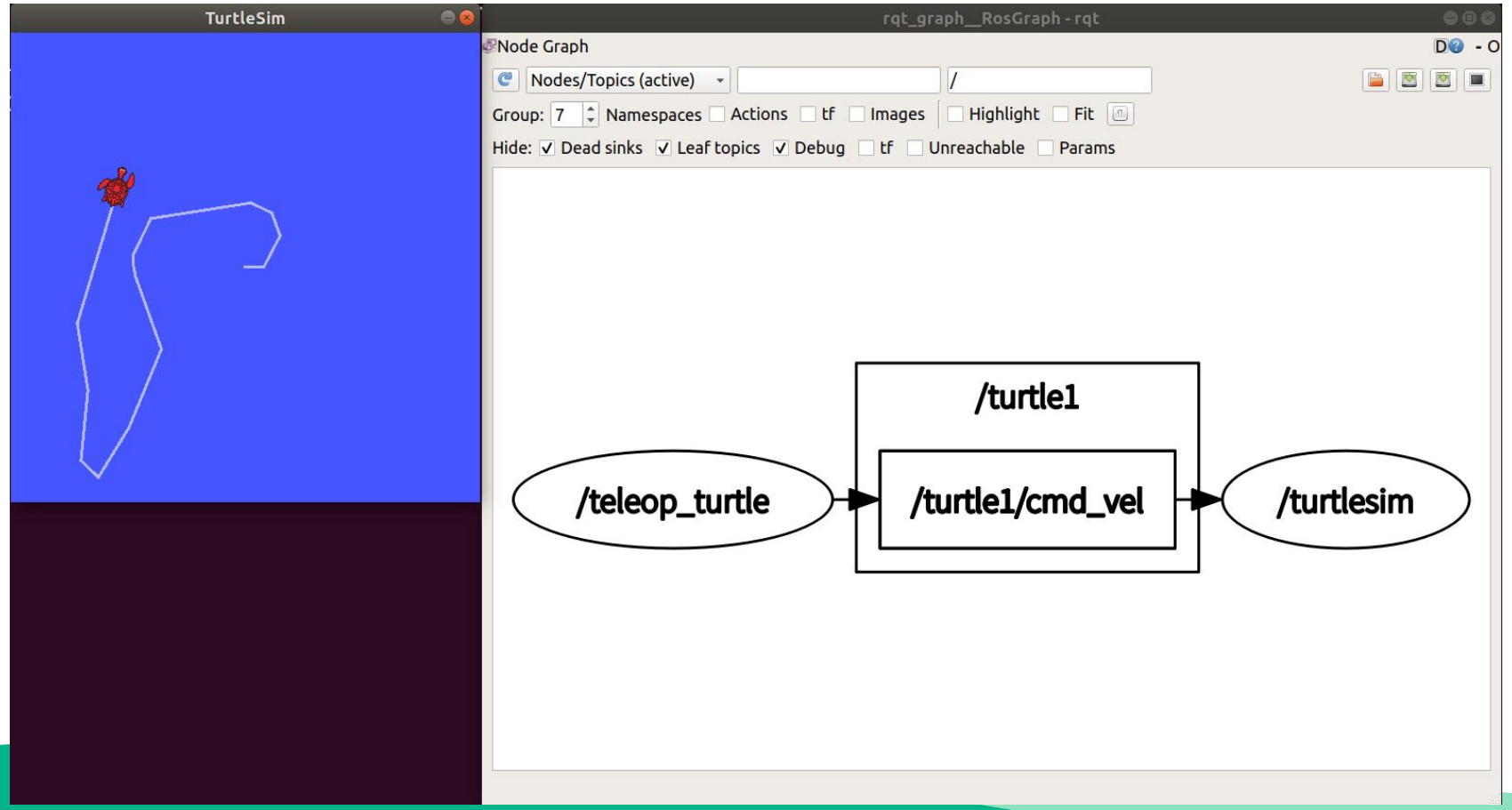
## 1.1.3 rqt plugins : rqt_graph

- **rqt_graph作用：可视化显示计算图**
- **rqt_graph启动：可以在 rqt 的 plugins 中添加，或者使用 rqt_graph 启动**



# 1.1.3 rqt plugins : rqt_graph

- 键盘操作海龟对应的 rqt_graph

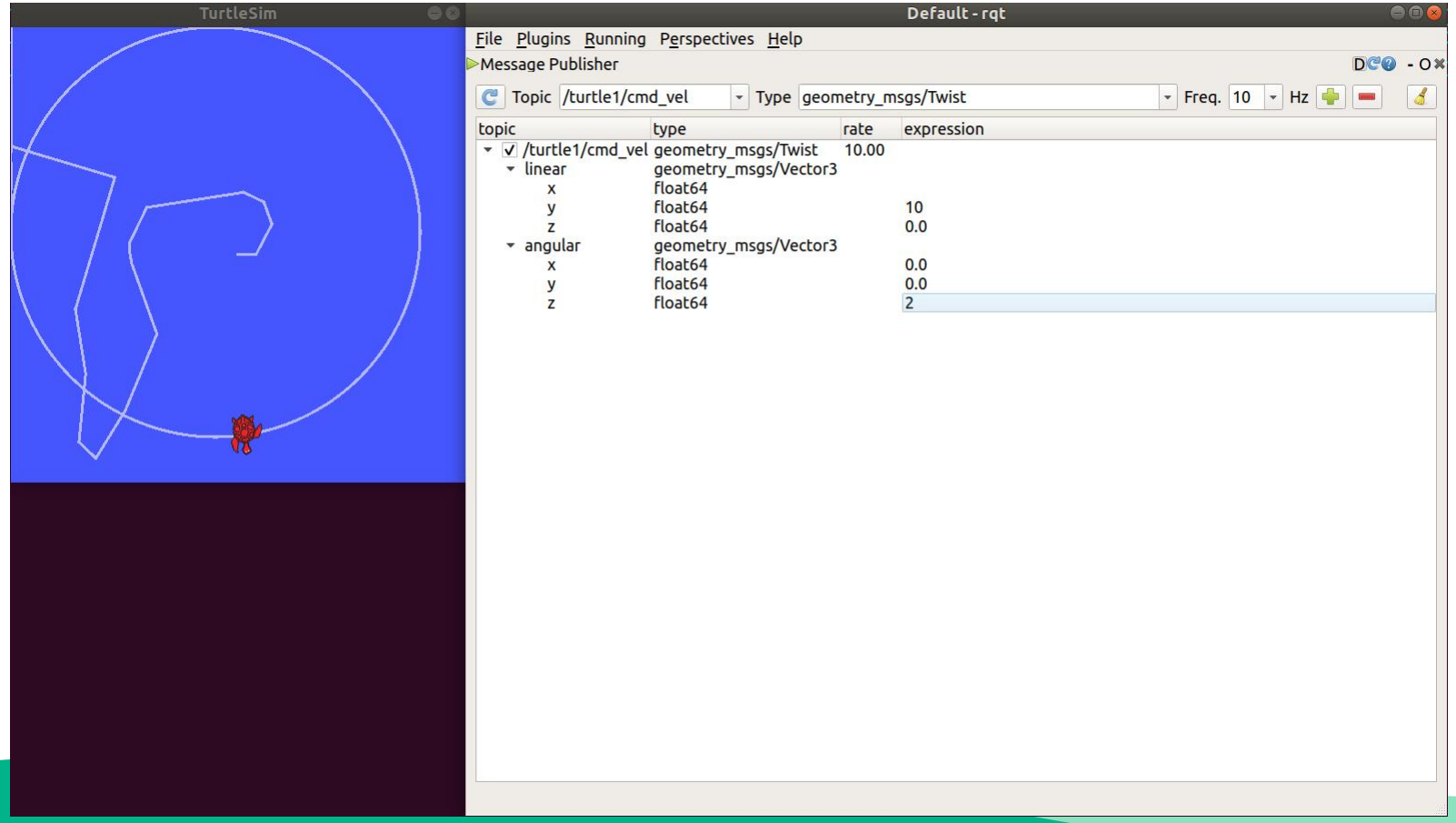


# 1.1 rqt

## 1.1.3 rqt plugins : rqt_publisher

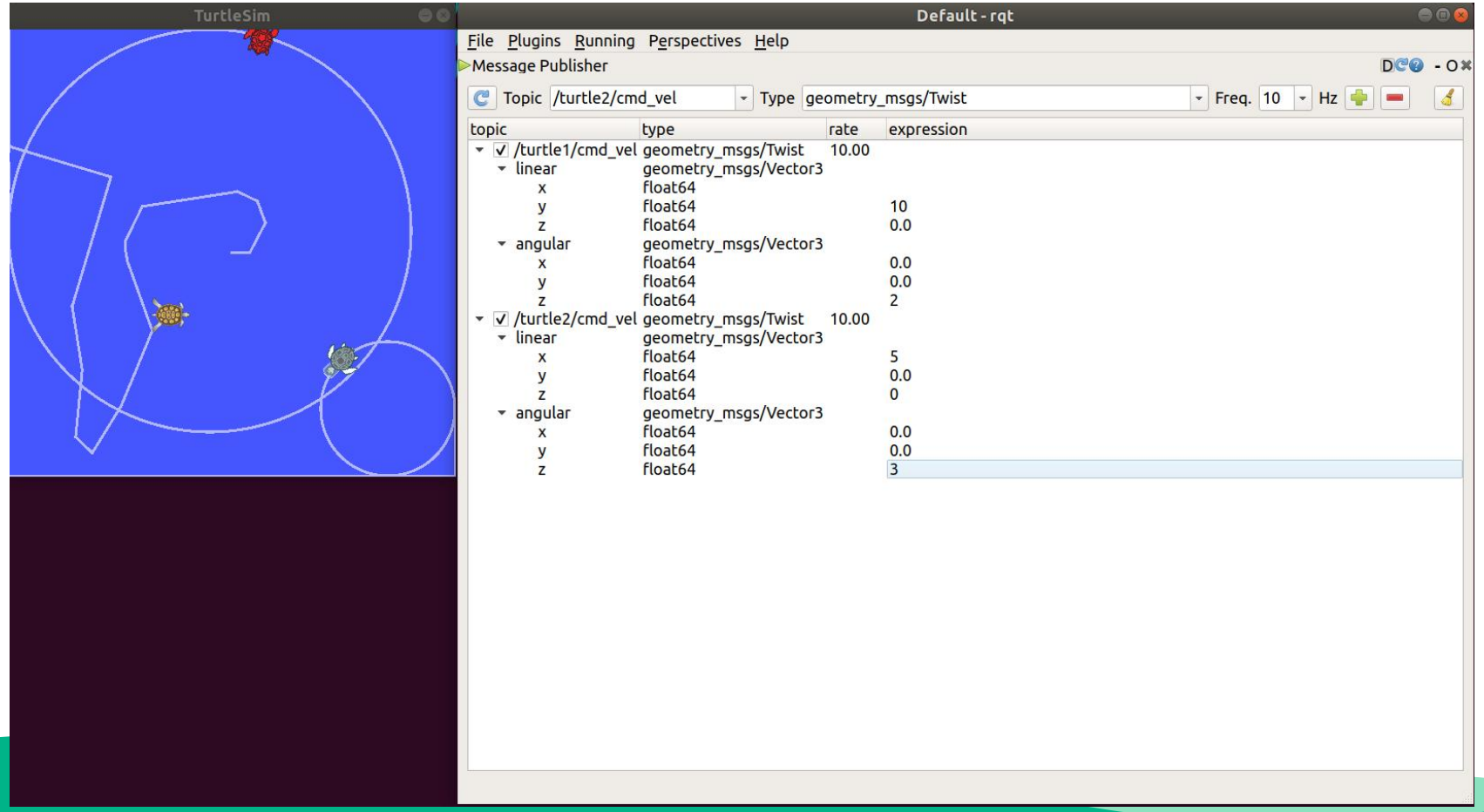
- 通过图形化界面操作海龟，命令行启动方法：

```
$ rosrun rqt_publisher rqt_publisher
```



# 1.1.3 rqt plugins : rqt_service_caller

- 通过图形化界面操作海龟，命令行启动方法：`$ rosrun rqt_service_caller rqt_service_caller`



## 1.1.3 rqt plugins : rqt_console

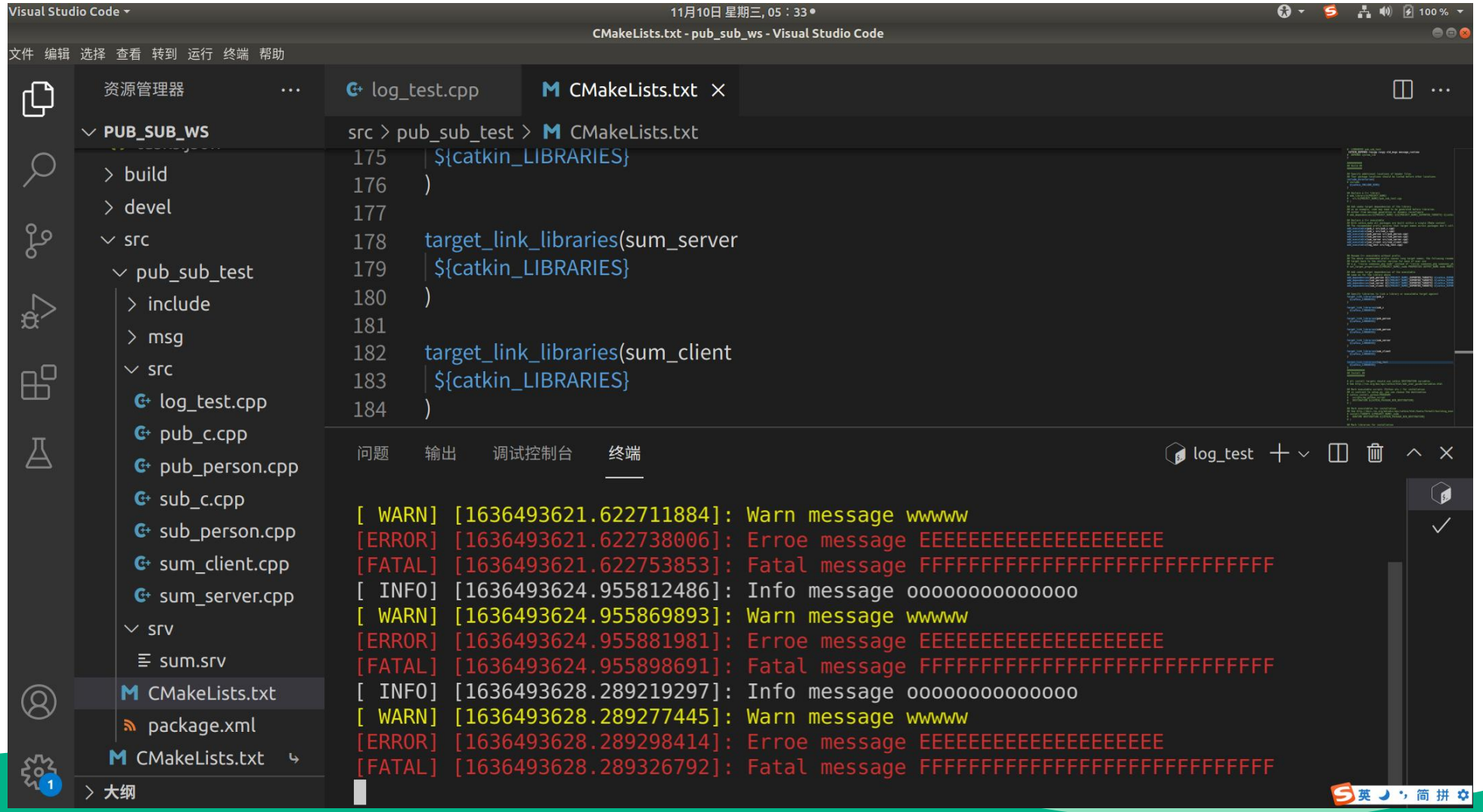
- **rqt_console作用：** rqt_console 是 ROS 中用于显示和过滤日志的图形化插件
- **rqt_console准备：** 编写 Node 节点输出各个级别的日志信息

```
#include "ros/ros.h"

int main(int argc, char *argv[])
{
    ros::init(argc,argv,"log_demo");
    ros::NodeHandle nh;
    ros::Rate r(1);
    while (ros::ok())
    {
        ROS_DEBUG("Debug message d");
        ROS_INFO("Info message ooooooooooooo");
        ROS_WARN("Warn message wwwww");
        ROS_ERROR("Erroe message EEEEEEEEEEEEEEEEEE");
        ROS_FATAL("Fatal message FFFFFFFFFFFFFFFFFFFFFFFFFF");
        r.sleep();
    }
    return 0;
}
```

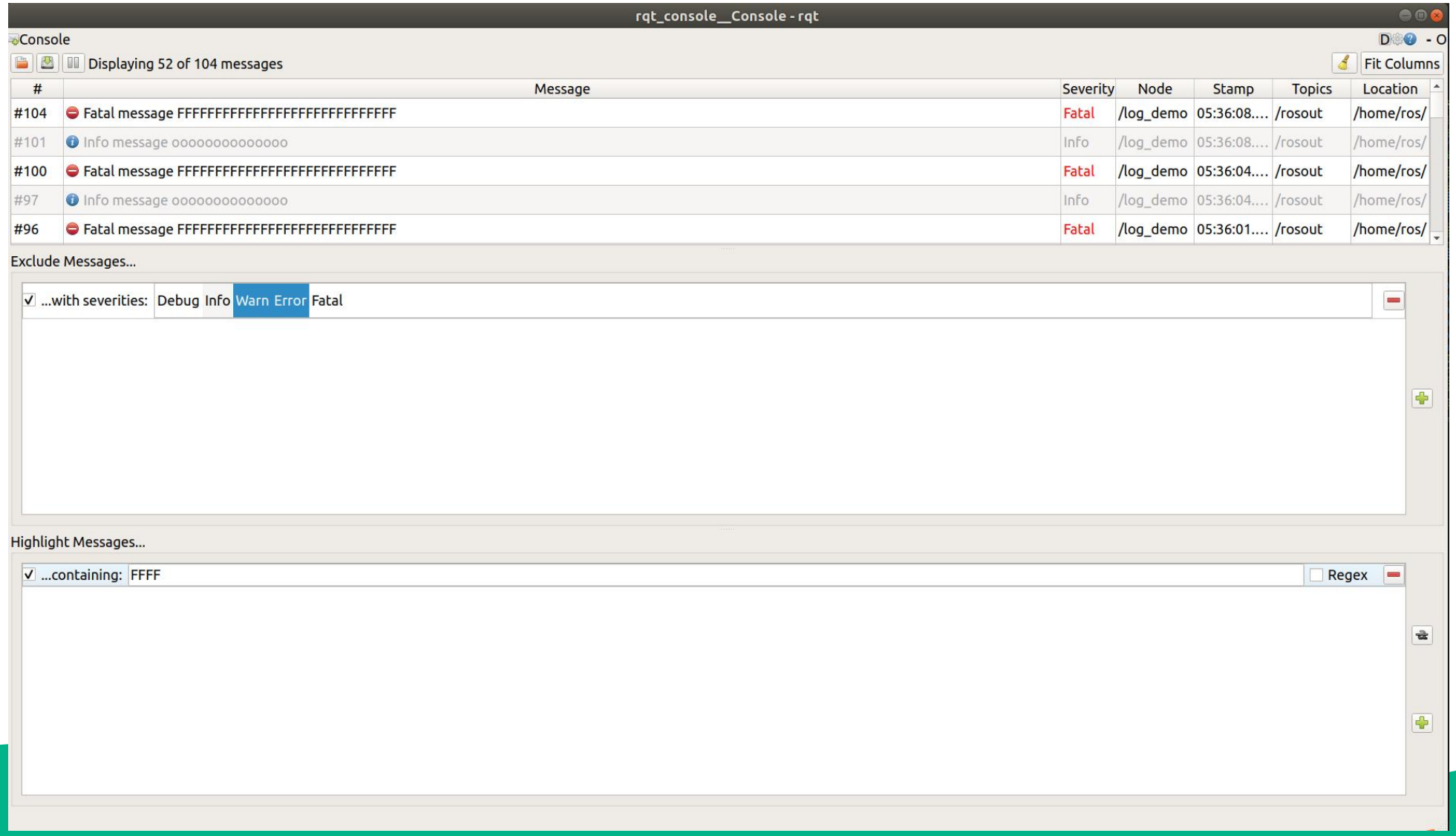
# 1.1 rqt

## 1.1.3 rqt plugins : rqt_console



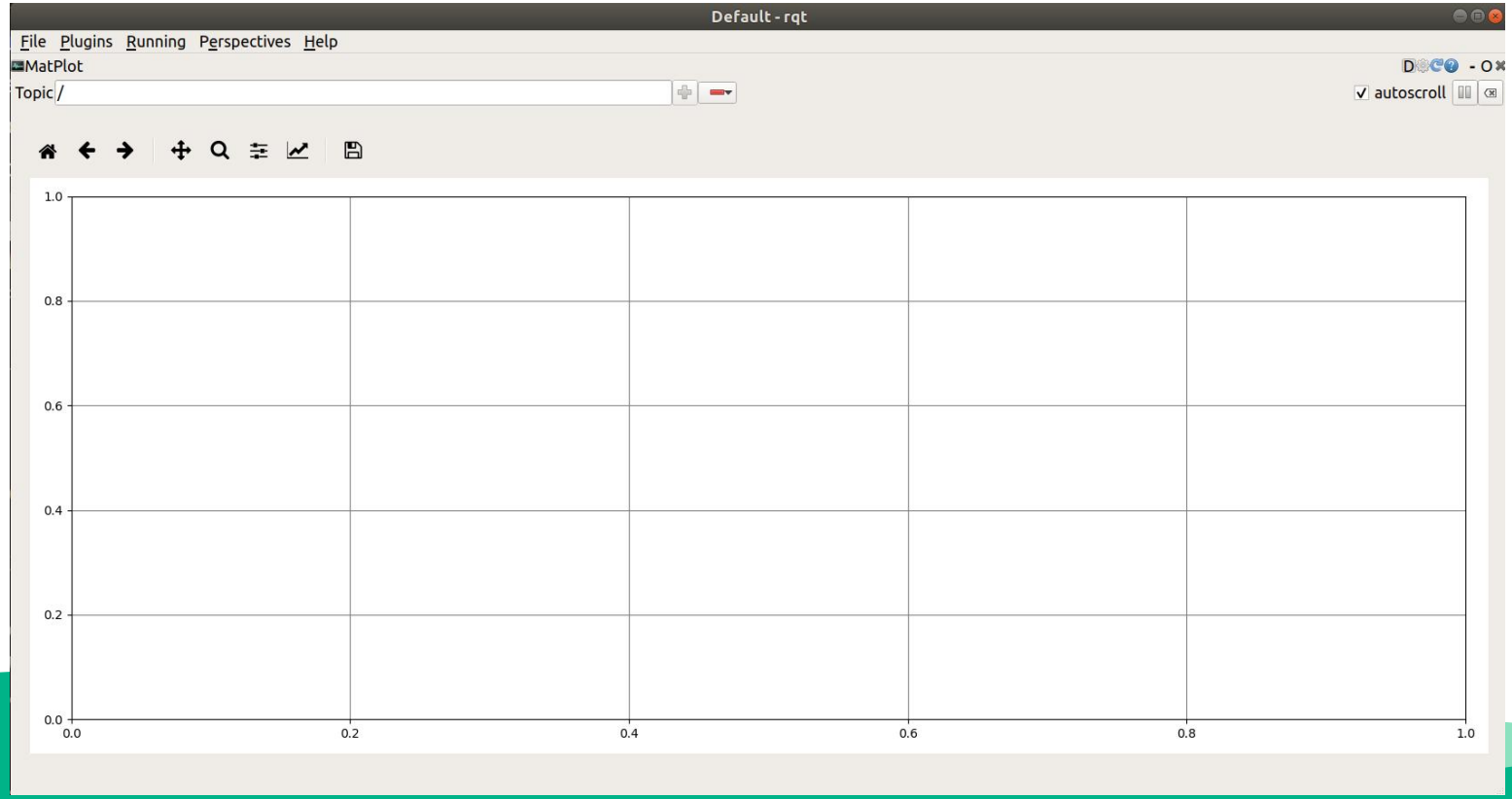
# 1.1 rqt

## 1.1.3 rqt plugins : rqt_console



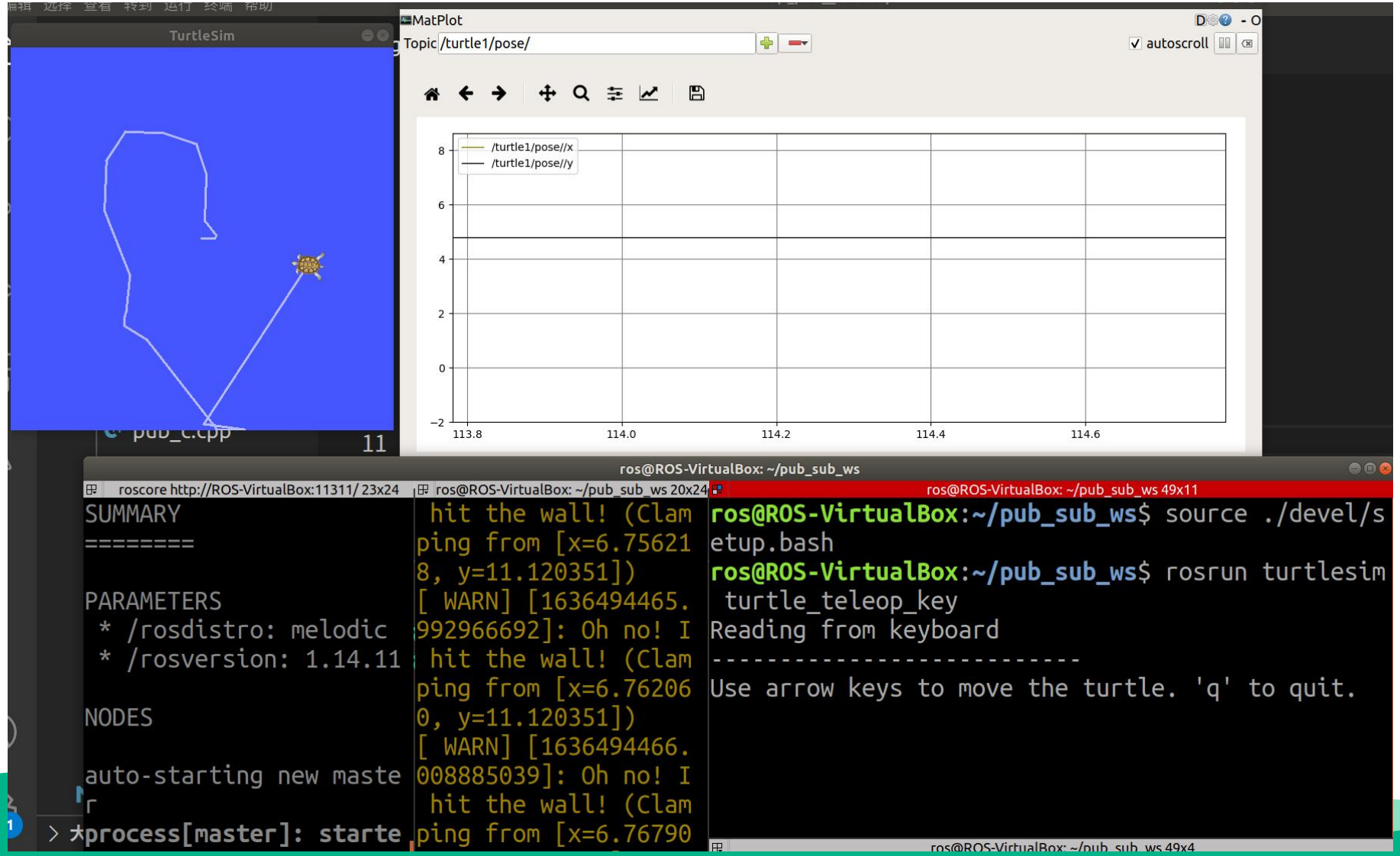
# 1.1.3 rqt plugins : rqt_plot

- **rqt_plot简介**：图形绘制插件，以 2D 绘图的方式绘制发布在 topic 上的数据
- **rqt_plot案例**：启动 turtlesim 乌龟节点与键盘控制节点，通过 rqt_plot 获取乌龟位姿
- **rqt_plot启动**：可以在 rqt 的 plugins 中添加，或者使用 rqt_plot 启动



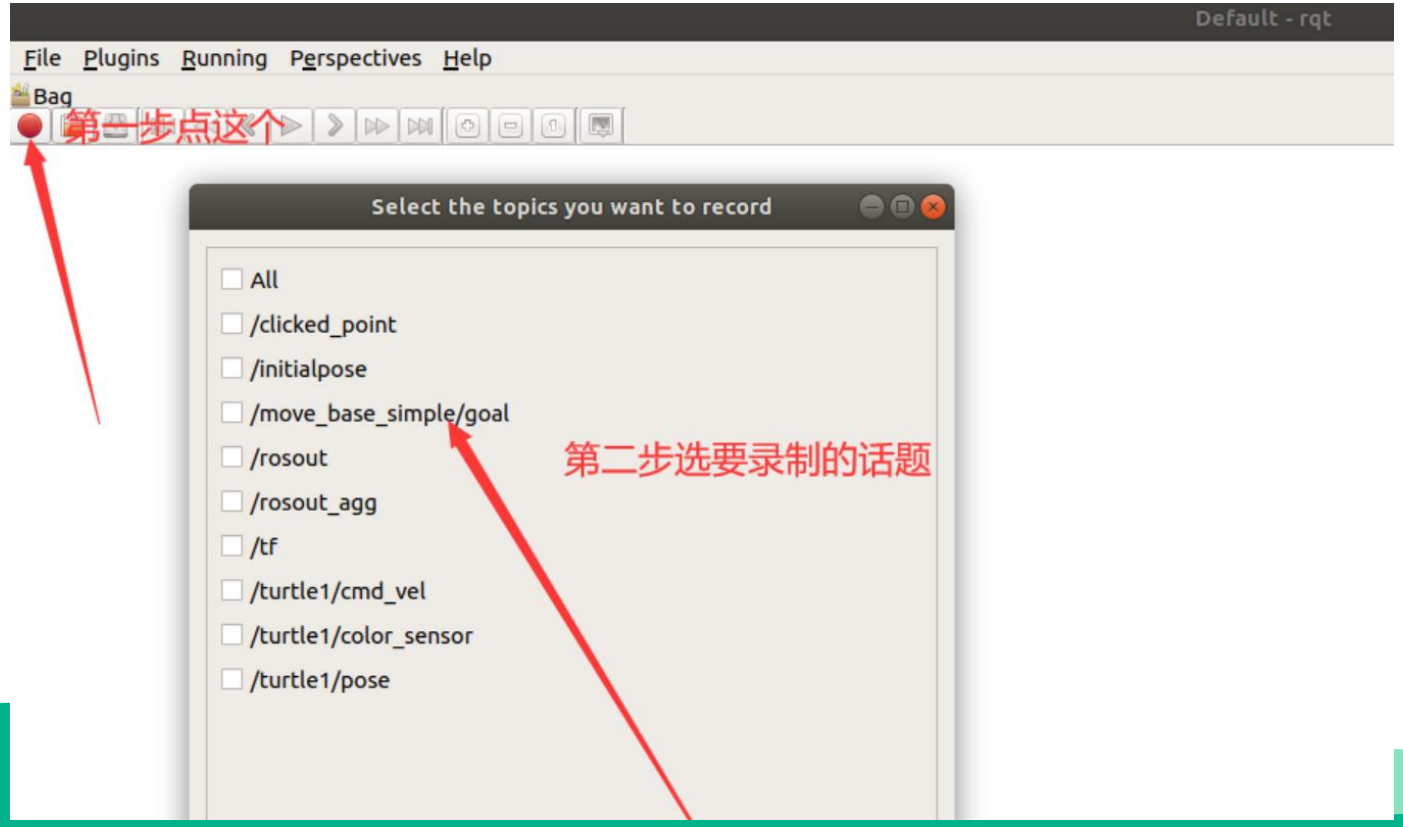
# 1.1 rqt

## 1.1.3 rqt plugins : rqt_plot



# 1.1.3 rqt plugins : rqt_bag

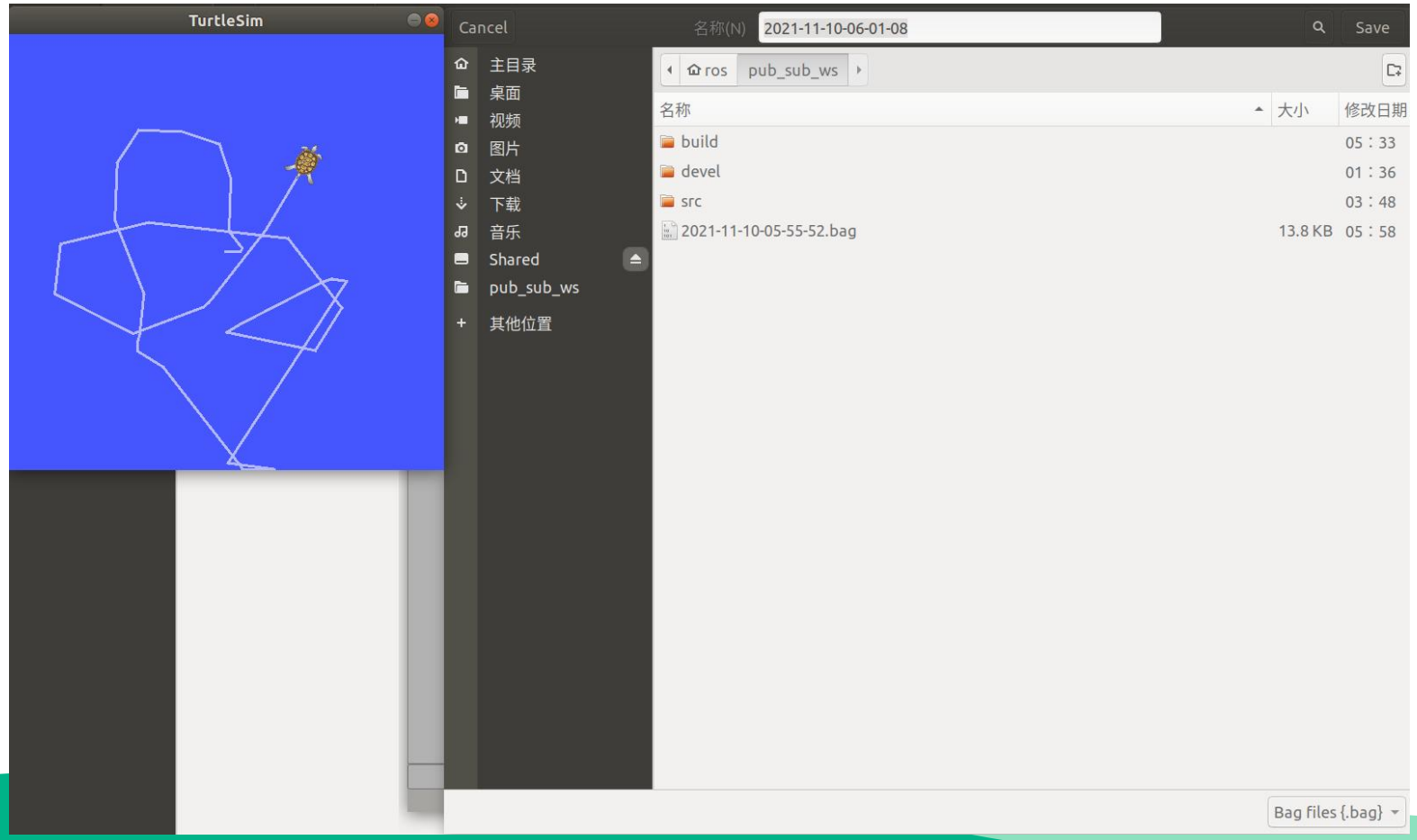
- **rqt_bag简介**：录制和重放 bag 文件的图形化插件
- **rqt_bag案例**：启动 turtlesim 乌龟节点与键盘控制节点
- **rqt_bag启动**：可以在 rqt 的 plugins 中添加，或者使用 rqt_bag 启动，选择需要录制的话题



# 1.1 rqt

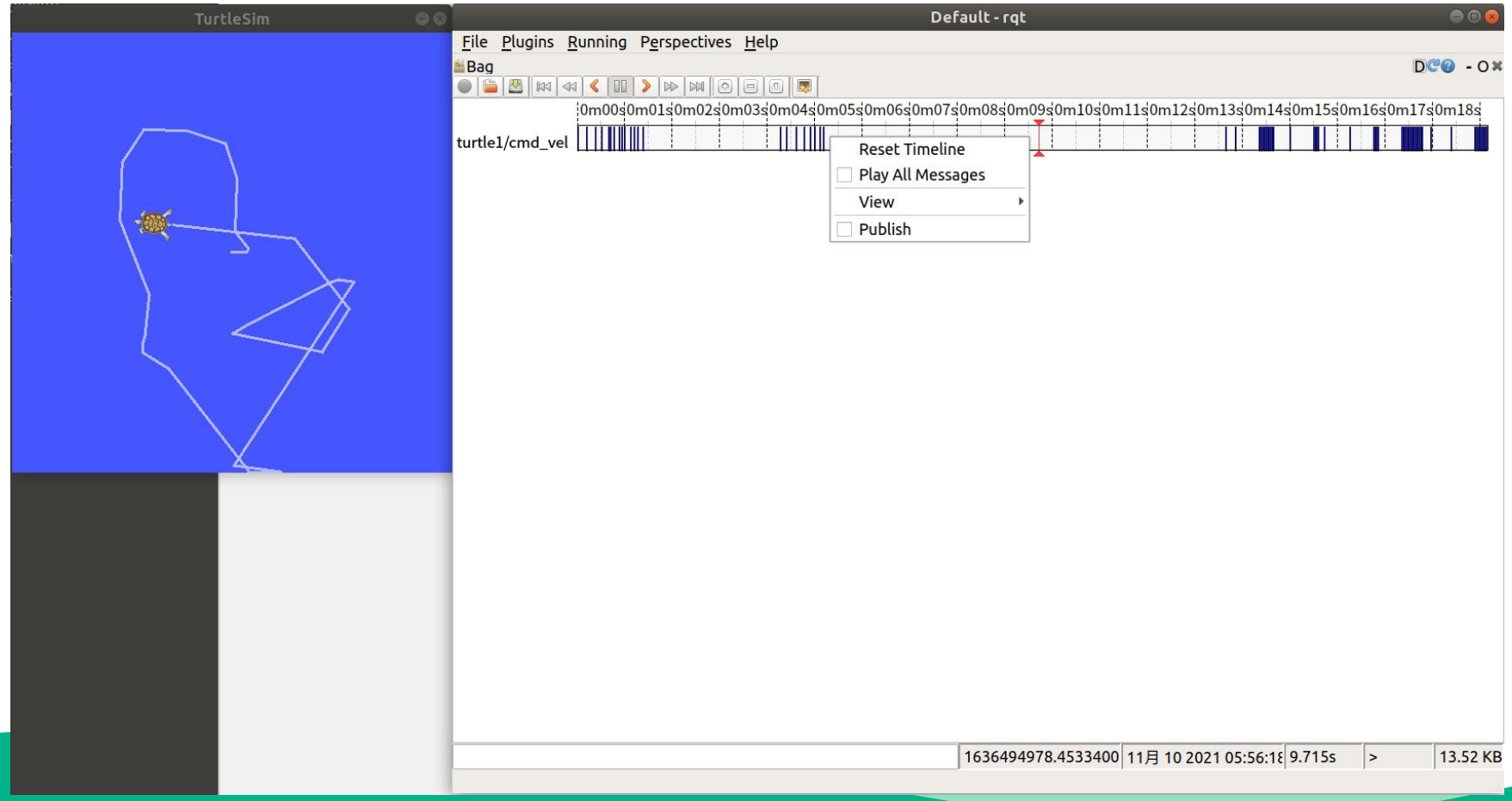
## 1.1.3 rqt plugins : rqt_bag

- **rqt_bag存储**：保存录制的 bag 文件



# 1.1.3 rqt plugins : rqt_bag

- rqt_bag结束：使用 ctrl + c 结束录制



# 1.1.3 rqt plugins : rqt_bag

- **rqtbag 命令行工具**

### 一、rosbag 基本作用

rosbag 工具可以录制一个包、从一个或多个包中重新发布消息、查看一个包的基本信息、检查一个包的消息定义，基于 Python 表达式过滤一个包的消息，压缩和解压缩一个包以及重建一个包的索引。

rosbag 目前常用的命令如下（fix 和 filter 暂时没有用到）：

- `record`：用指定的话题录制一个 bag 包
- `info`：显示一个 bag 包的基本信息，比如包含哪些话题
- `play`：回放一个或者多个 bag 包
- `check`：检查一个 bag 包在当前的系统中是否可以回放和迁移
- `compress`：压缩一个或多个 bag 包
- `decompress`：解压缩一个或多个 bag 包
- `reindex`：重新索引一个或多个损坏 bag 包

我在目前项目的使用中，用的最多的就是 record、info 以及 play 功能，先录制想要的话题包，录制完毕检查下包的信息，最后再回放作为算法的输入，下面就详细学习下上面命令的用法。

# 1.1.3 rqt plugins : rqt_bag

- **rqtbag 命令行工具**

**1. 录包**

录制所有话题：

```
rosbag record -a
```

录制指定话题，设置 bag 包名：

```
rosbag record -O bag_name.bag /topic1_name /topic2_name /xxx
```

有时候我录制包不设置名称，默认按照录制结束时间命名：

```
rosbag record /topic1_name /topic2_name /xxx
```

# 1.1.3 rqt plugins : rqt_bag

- **rqtbag 命令行工具**

**2. 回放**

我常用的是以暂停的方式启动，防止跑掉数据：

```
rosbag play --pause record.bag
```

你也可以直接回放：

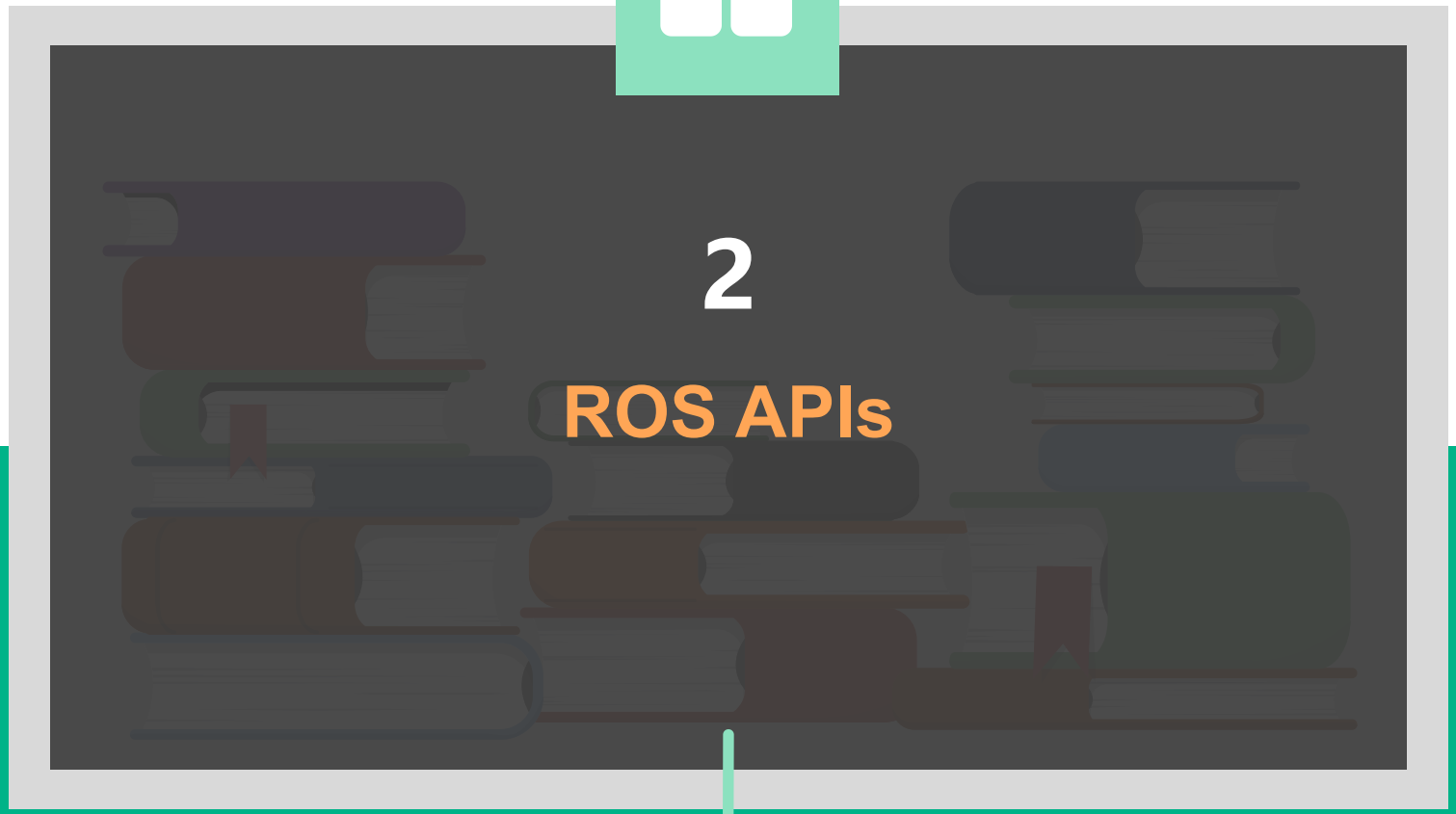
```
rosbag play record.bag
```

我建图过程中有时会设置以 0.5 倍速回放，也就是以录制频率的一半回放：

```
rosbag play -r 0.5 record.bag
```

回放完后，我一般会用 `rostopic list` 查看下发布的主题，确保是我需要的：

```
rostopic list
```



# 2

# ROS APIs

# 2.1 ROS APIs

## 2.1.1 Overview

- API（Application Programming Interface，应用程序接口）是一些预先定义的接口（如函数、HTTP接口），或指软件系统不同组成部分衔接的约定
- API 用来提供应用程序与开发人员基于某软件或硬件得以访问的一组例程，无需访问软件源码或理解内部工作机制即可使用软件的相关功能
- 官方API文档 : http://wiki.ros.org/APIs
  1. ROS节点的初始化相关API
  2. NodeHandle的基本使用相关API
  3. 话题的发布方，订阅方相关API
  4. 服务的服务端，客户端相关API
  5. 时间相关API
  6. 日志输出相关API
  7. 参数服务器相关API

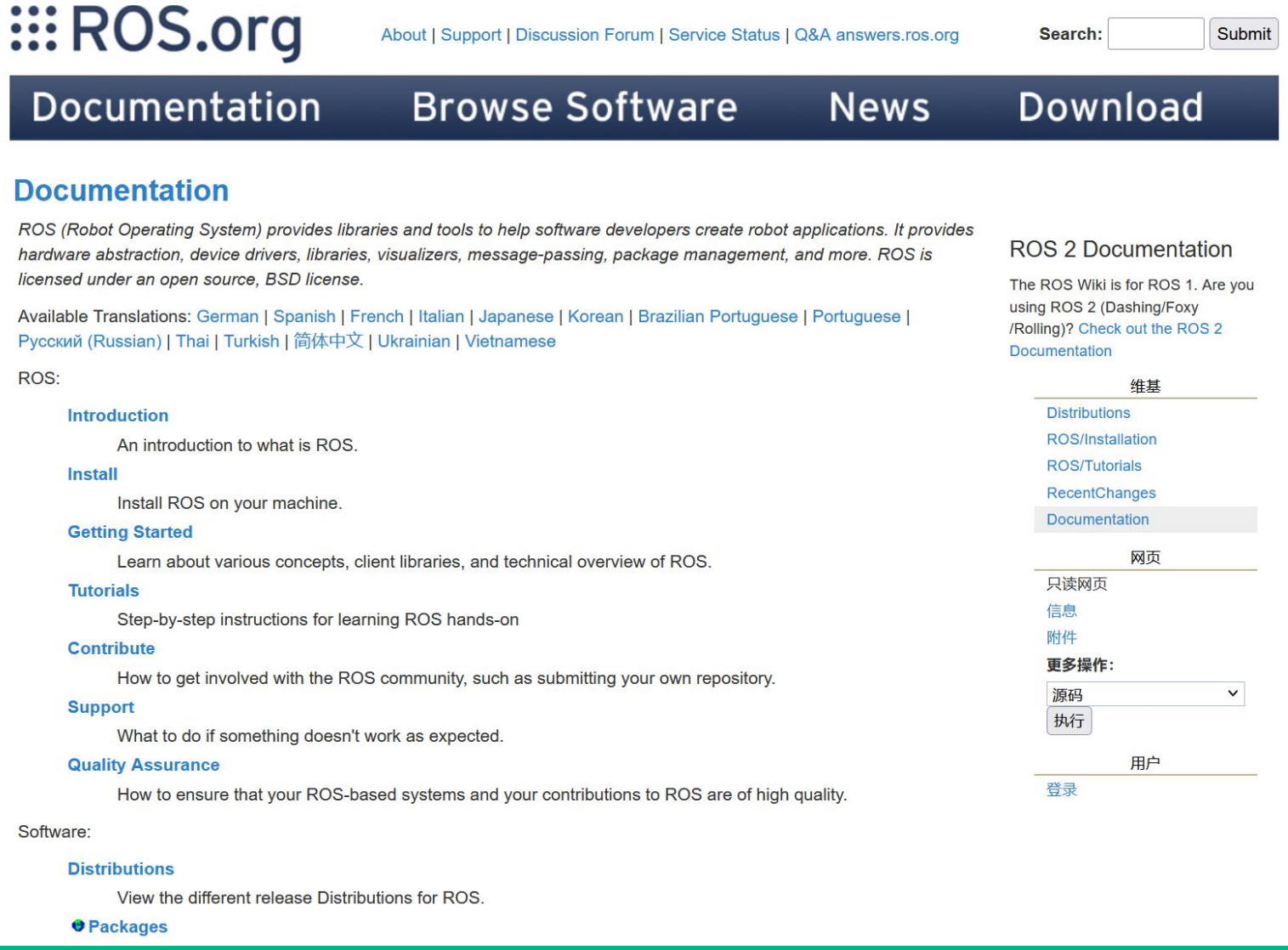
# 2.1 ROS APIs

## 2.1.1 Overview

| API | ROS | C++ | Python |
|---|---|---|---|
| ROS | ROS | roscpp | rospy |
| Basic Datatypes | common_msgs | common_msgs | common_msgs |
| Manipulating message streams | topic_tools | message_filters | message_filters |
| Drivers | joystick_drivers, camera_drivers, laser_drivers, audio_common (aka sound_drivers), imu_drivers | joystick_drivers, camera_drivers, laser_drivers, audio_common, imu_drivers | |
| Driver Implementation | driver_common | driver_common | |
| Filtering data | | filters | |
| 3D processing | laser_pipeline, perception_pcl | laser_pipeline, perception_pcl | |
| Image processing | | image_common, image_pipeline, vision_opencv | vision_opencv |
| Transforms/Coordinates | | tf, tf_conversions, robot_state_publisher | tf, tf_conversions |
| Actions | actionlib | actionlib | actionlib |
| Executive/Task Manager | executive_smach | | executive_smach |
| Navigation | navigation | via actionlib | via actionlib |
| Simulation (2D) | simulator_stage | simulator_stage | |
| Simulation (3D) | simulator_gazebo | simulator_gazebo | |
| Robot Model | | robot_model | |
| Realtime Controllers | pr2_controller_manager | pr2_controller_interface, realtime_tools | |
| Motion planning (arms) | ompl, chomp_motion_planner, sbpl | actionlib through move_arm | actionlib through move_arm |
| Humanoid walk | walk_msgs | walk_interfaces | |

# 2.1 ROS APIs

## 2.1.2 Common APIs



ROS.org

About | Support | Discussion Forum | Service Status | Q&A answers.ros.org

Search: Submit

Documentation Browse Software News Download

### Documentation

*ROS (Robot Operating System) provides libraries and tools to help software developers create robot applications. It provides hardware abstraction, device drivers, libraries, visualizers, message-passing, package management, and more. ROS is licensed under an open source, BSD license.*

Available Translations: German | Spanish | French | Italian | Japanese | Korean | Brazilian Portuguese | Portuguese | Русский (Russian) | Thai | Turkish | 简体中文 | Ukrainian | Vietnamese

ROS:

**Introduction**
An introduction to what is ROS.

**Install**
Install ROS on your machine.

**Getting Started**
Learn about various concepts, client libraries, and technical overview of ROS.

**Tutorials**
Step-by-step instructions for learning ROS hands-on

**Contribute**
How to get involved with the ROS community, such as submitting your own repository.

**Support**
What to do if something doesn't work as expected.

**Quality Assurance**
How to ensure that your ROS-based systems and your contributions to ROS are of high quality.

Software:

**Distributions**
View the different release Distributions for ROS.

**Packages**

ROS 2 Documentation

The ROS Wiki is for ROS 1. Are you using ROS 2 (Dashing/Foxy /Rolling)? Check out the ROS 2 Documentation

维基
Distributions
ROS/Installation
ROS/Tutorials
RecentChanges
Documentation

网页
只读网页
信息
附件
**更多操作:**
源码
执行

用户
登录

# 2.1 ROS APIs

## 2.1.2 Common APIs

### common_msgs

kinetic | melodic | noetic | *Show EOL distros:* ☐

Documentation Status

***common_msgs**: actionlib_msgs | diagnostic_msgs | geometry_msgs | nav_msgs | sensor_msgs | shape_msgs | stereo_msgs | trajectory_msgs | visualization_msgs*

## Package Summary

Released | Continuous Integration: 3 / 3 | Documented

common_msgs contains messages that are widely used by other ROS packages. These includes messages for actions (actionlib_msgs), diagnostics (diagnostic_msgs), geometric primitives (geometry_msgs), robot navigation (nav_msgs), and common sensors (sensor_msgs), such as laser range finders, cameras, point clouds.

- Maintainer status: maintained
- Maintainer: Michel Hidalgo <michel AT ekumenlabs DOT com>
- Author: Tully Foote <tfoote AT osrfoundation DOT org>
- License: BSD
- Source: git https://github.com/ros/common_msgs.git (branch: noetic-devel)

**目录**

1. Message and Service Types
2. Standard Units of Measure and Coordinate Conventions
3. Purpose of common_msgs stack
4. Guidelines for submitting to common_msgs
5. Packages within common_msgs
    1. actionlib_msgs
    2. diagnostic_msgs
    3. geometry_msgs
    4. nav_msgs
    5. sensor_msgs
6. Migrating Messages
7. Report a Bug

**Package Links**

Tutorials
Troubleshooting
FAQ
Changelog
Change List
Reviews

**Dependencies (10)**
**Used by (1)**
**Jenkins jobs (10)**

ROS 2 Documentation

The ROS Wiki is for ROS 1. Are you using ROS 2 (Dashing/Foxy/Rolling)? Check out the ROS 2 Documentation

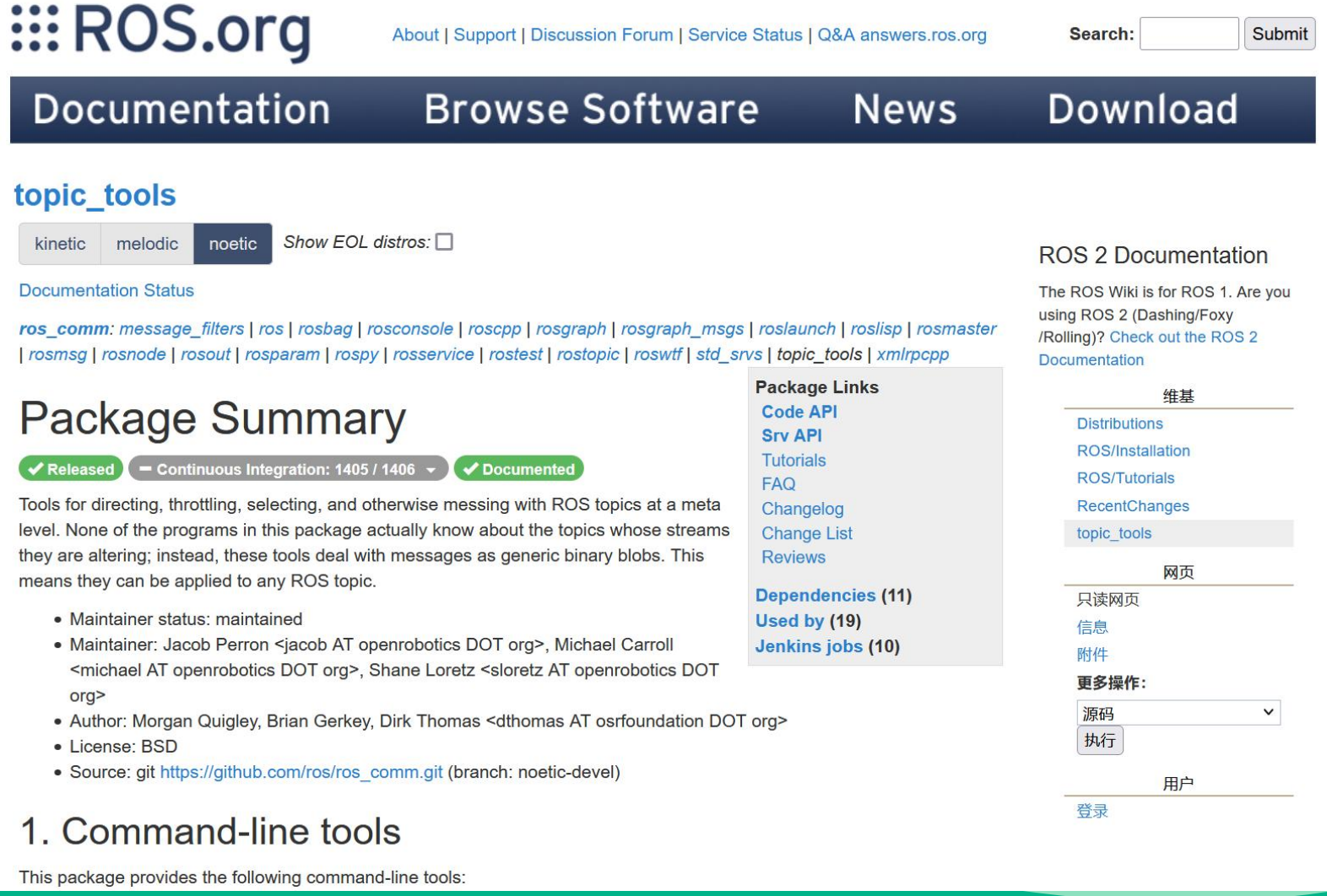
维基
Distributions
ROS/Installation
ROS/Tutorials
RecentChanges
common_msgs

网页
只读网页
信息
附件
**更多操作:**
源码
执行

用户
登录

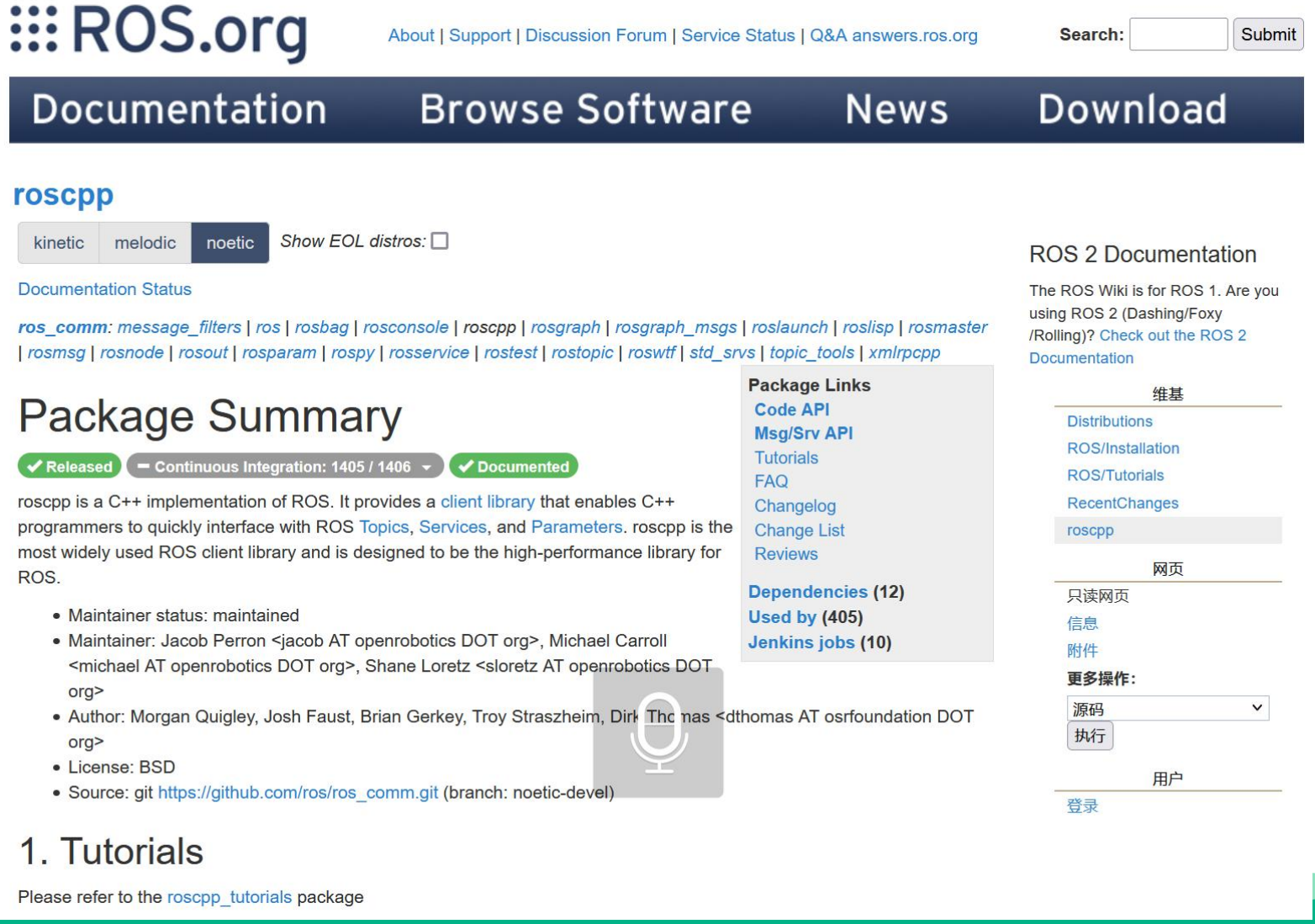
# 2.1 ROS APIs

## 2.1.2 Common APIs



# 2.1 ROS APIs

## 2.1.3 C++ APIs



# 2.1 ROS APIs

## 2.1.3 C++ APIs

Main Page | Related Pages | Namespaces | Classes | Files

### roscpp

**roscpp** is a ROS client implementation in C++. The main parts of **roscpp** are:

- **ros::init()** : A version of **ros::init()** must be called before using any of the rest of the ROS system.
- **ros::NodeHandle** : Public interface to topics, services, parameters, etc.
- **ros::master** : Contains functions for querying information from the master
- **ros::this_node** : Contains functions for querying information about this process' node
- **ros::service** : Contains functions for querying information about services
- **ros::param** : Contains functions for querying the parameter service without the need for a **ros::NodeHandle**
- **ros::names** : Contains functions for manipulating ROS graph resource names

**Examples**

Many examples of using ROS can be found on the wiki and in the roscpp_tutorials package.

**roscpp**

*roscpp is a C++ implementation of ROS. It provides a client library that enables C++ programmers to quickly interface with ROS Topics, Services, and Parameters. roscpp is the most widely used ROS client library and is designed to be the high-performance library for ROS.*

- Homepage: http://wiki.ros.org/roscpp

roscpp
Author(s): Morgan Quigley, Josh Faust, Brian Gerkey, Troy Straszheim, Dirk Thomas
autogenerated on Thu Sep 23 2021 03:02:04

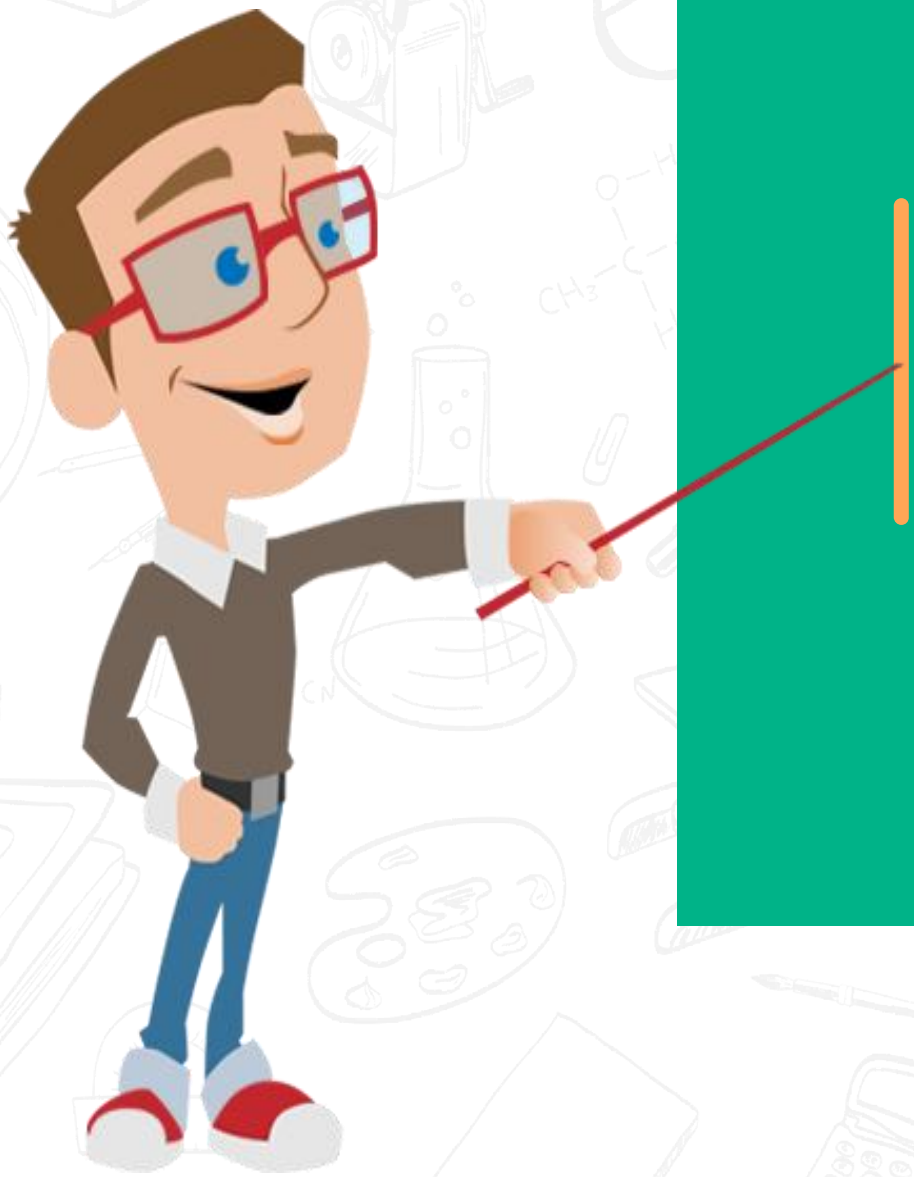
# Summary

1. rqt tools
    a) rqt_graph
    b) rqt_publisher
    c) rqt_service_caller
    d) rqt_console
    e) rqt_plot
    f) rqt_bag
2. ROS APIs
    a) ROS APIs
    b) C++ APIs

# Q&A

今日事今日毕

Question?



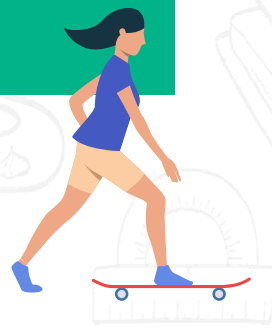
# 机器人操作系统（双语）

## Robot Operating System (Bilingual)

WHERE THERE'S A WILL, THERE'S A WAY

授课教师：郄(qiè)龙飞

机器人工程



# Overview lesson 11-2

1 Coordinate Transformation

2 Data Visualization Tool

3 ~

4 ~

# Objectives

1、理解 TF2 的基本概念

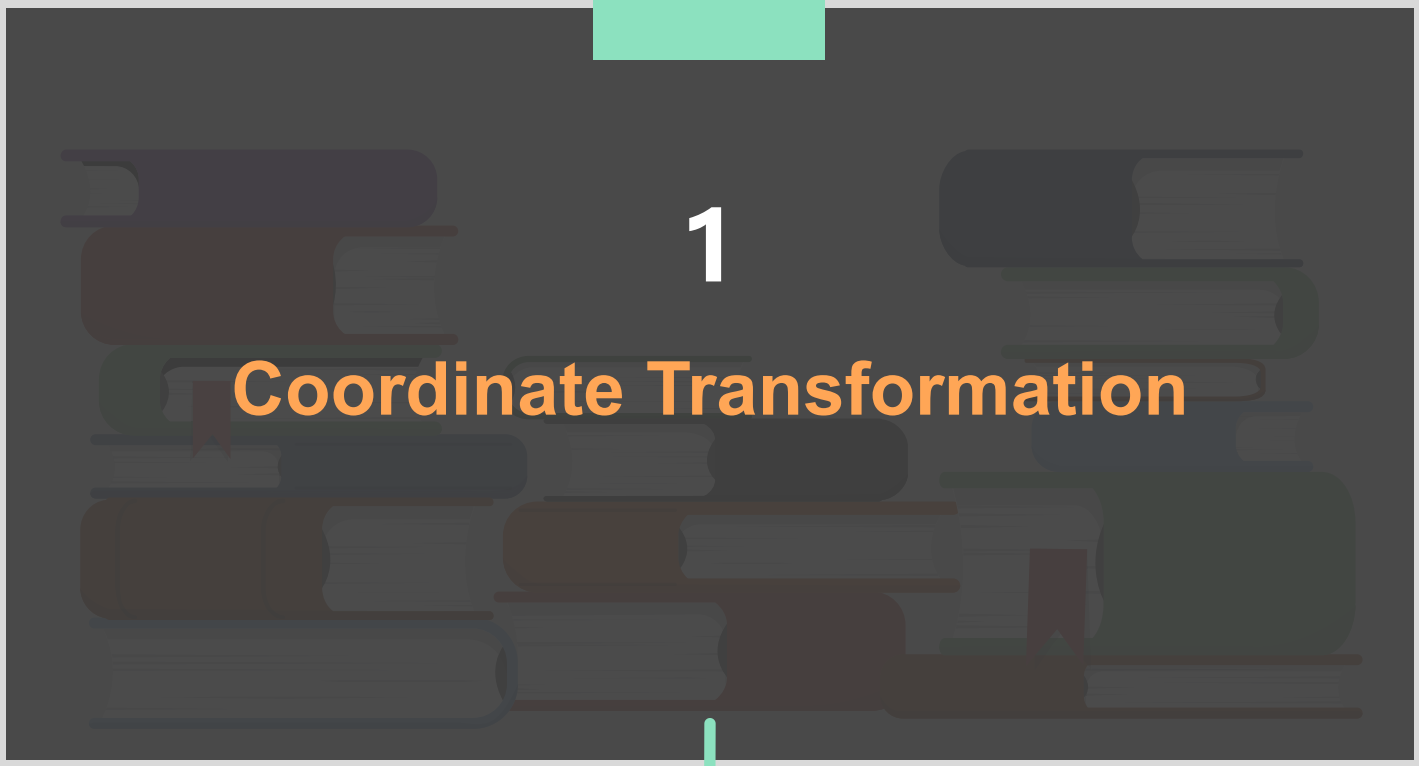
2、掌握 Rviz 的基本使用方法



# Pre-assessment

1、是否了解坐标变换的含义?

2、是否使用过Rviz?



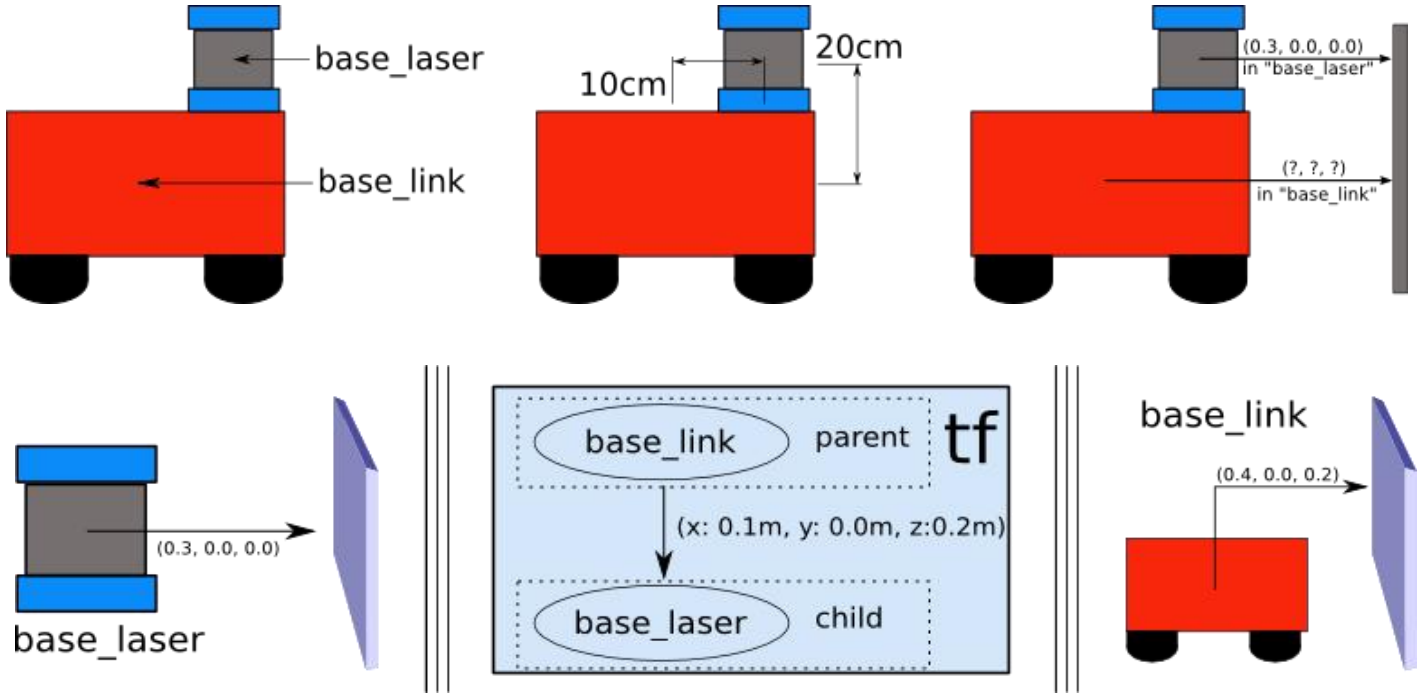
# 1

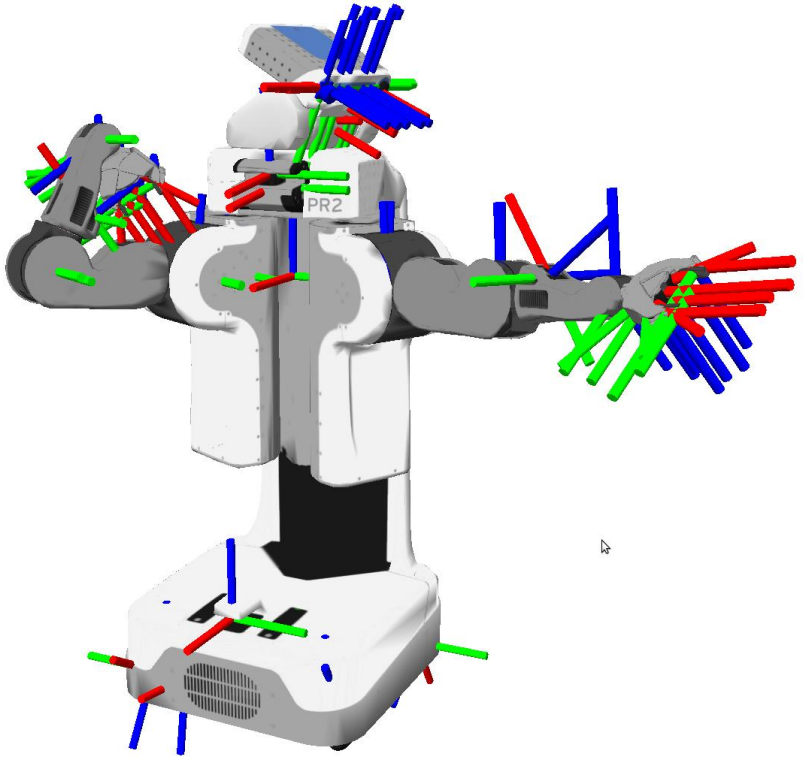
# Coordinate Transformation

# 1.1 TF2

## 1.1.1 Overview

- 为何需要坐标变换？什么情况需要使用坐标变换？



图中：红色，X+，绿色，Y+，蓝色，Z+

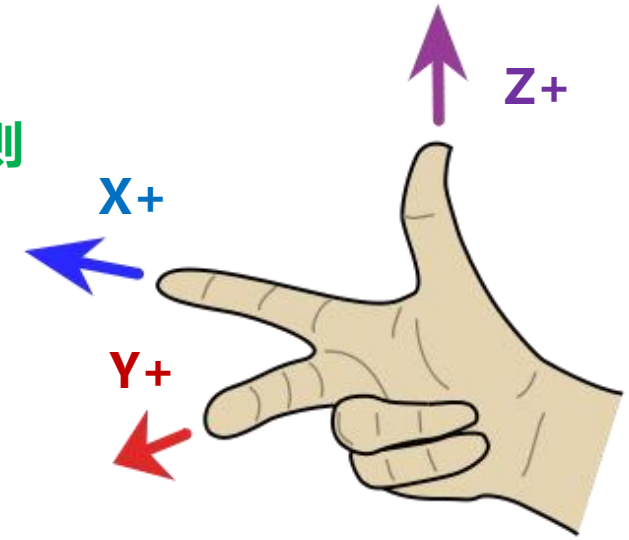
# 1.1 TF2

## 1.1.1 Overview

- TF(TransForm Frame)官方参考文档：http://wiki.ros.org/tf2
- tf is a package that lets the user keep track of multiple coordinate frames over time. tf maintains the relationship between coordinate frames in a tree structure buffered in time, and lets the user transform points, vectors, etc between any two coordinate frames at any desired point in time.
- tf2 is the second generation of the transform library, which lets the user keep track of multiple coordinate frames over time. tf2 maintains the relationship between coordinate frames in a tree structure buffered in time, and lets the user transform points, vectors, etc between any two coordinate frames at any desired point in time.

# 1.1 TF2

## 1.1.1 Overview




- **概念**：TF2以树的形式组织，ROS 中通过坐标系统进行物体标定，使用**右手定则**
- **作用**：在 ROS 中用于实现不同坐标系之间的**点**或**向量**的转换
- **案例**：乌龟跟随
- **补充说明**：
  - 在 ROS 中坐标变换最初对应的是 tf，从 hydro 版本开始, tf 被弃用，迁移到 tf2，后者更为简洁高效，tf2对应的常用功能包有：
    - tf2_geometry_msgs：可以将 ROS 消息转换成 tf2 消息
    - tf2：封装了坐标变换的常用消息
    - tf2_ros：为 tf2 提供了 roscpp 和 rospy 绑定，封装了坐标变换常用的APIs

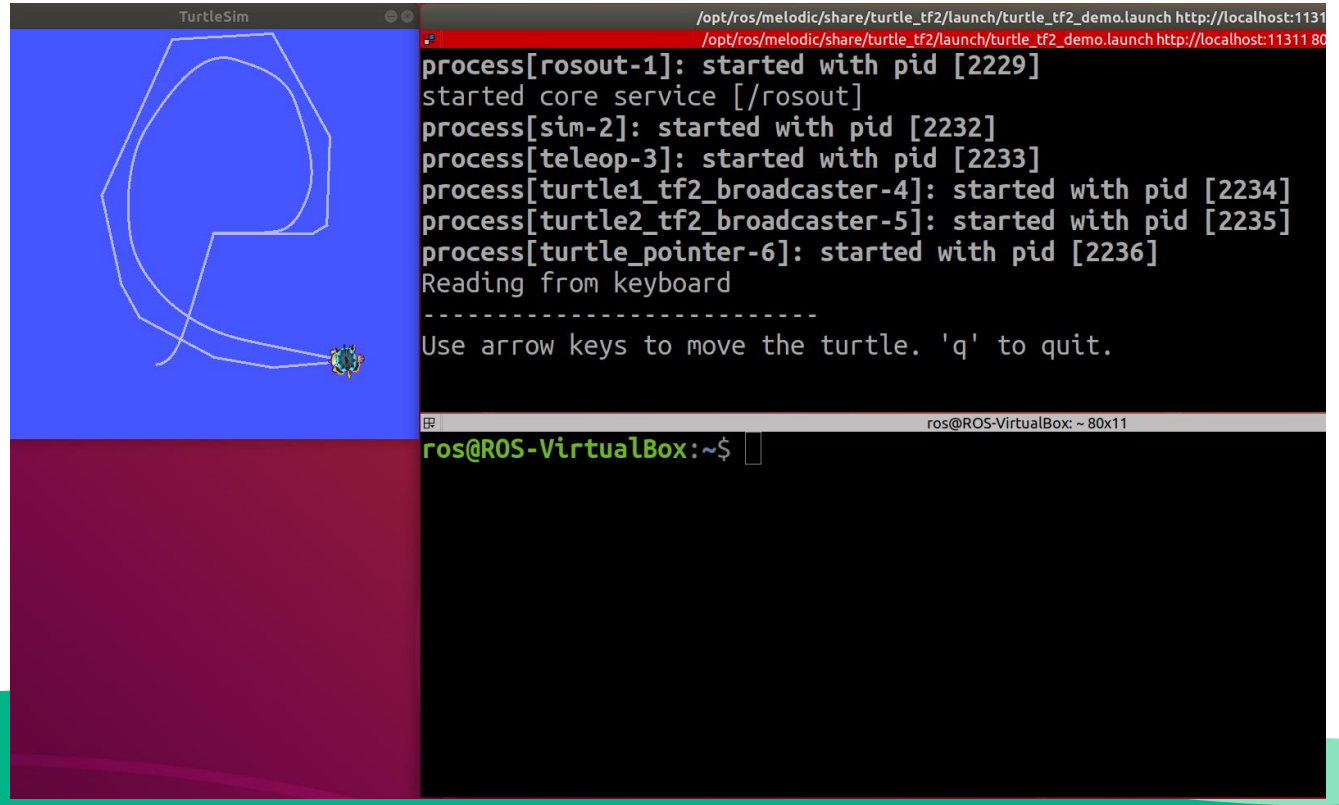
# 1.1 TF2

## 1.1.1 Overview

- 安装：

```
$ sudo apt install ros-noetic-turtle-tf2 ros-noetic-tf2-tools ros-noetic-tf
```

```
$ roslaunch turtle_tf2 turtle_tf2_demo_cpp.launch
```

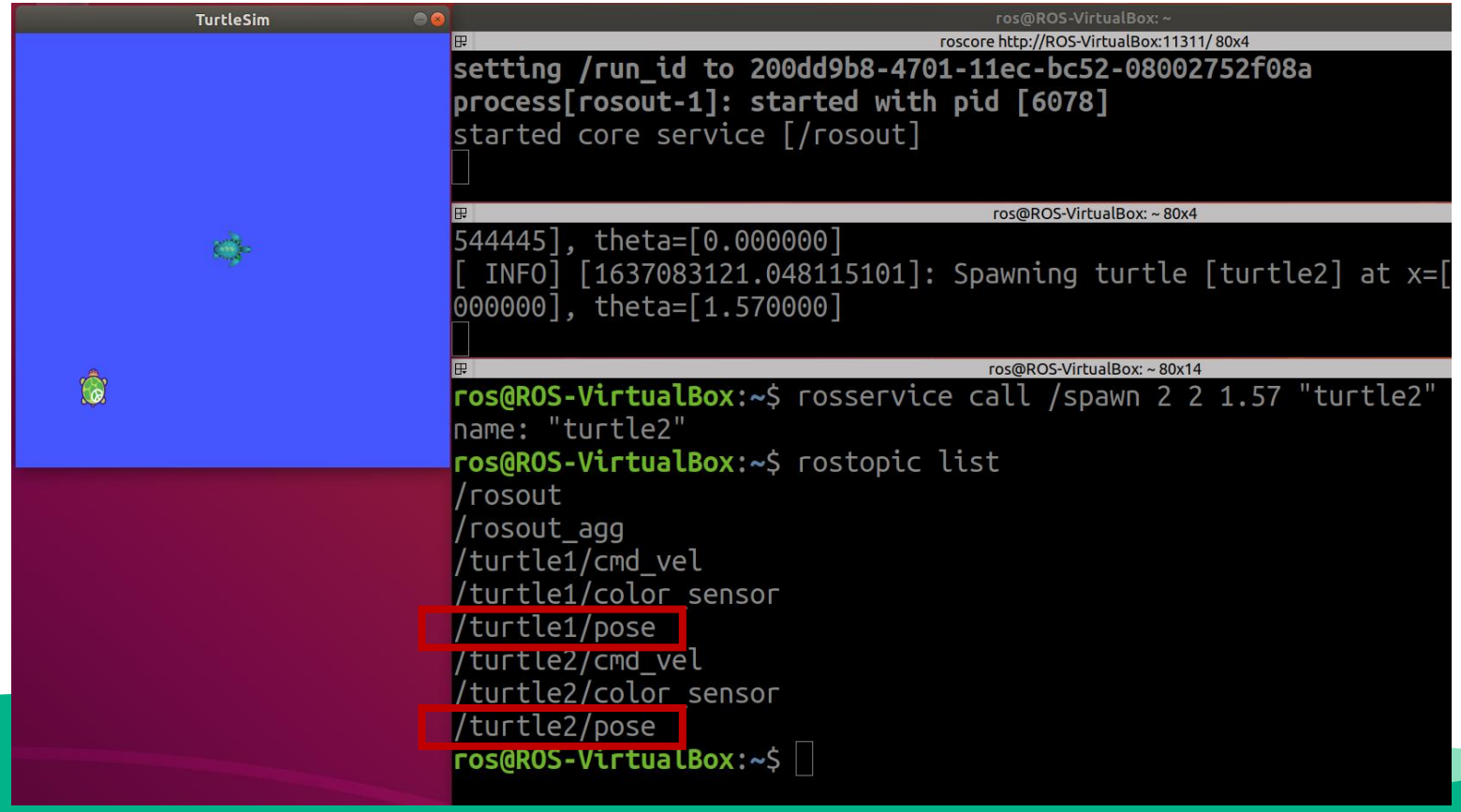


# 1.1 TF2

## 1.1.1 Overview

- 实现流程分析：

1. 启动乌龟显示节点，生成乌龟1；调用服务，在乌龟显示窗体中生成乌龟2，此时会有乌龟1和乌龟2的位姿信息



## 1.1.1 Overview

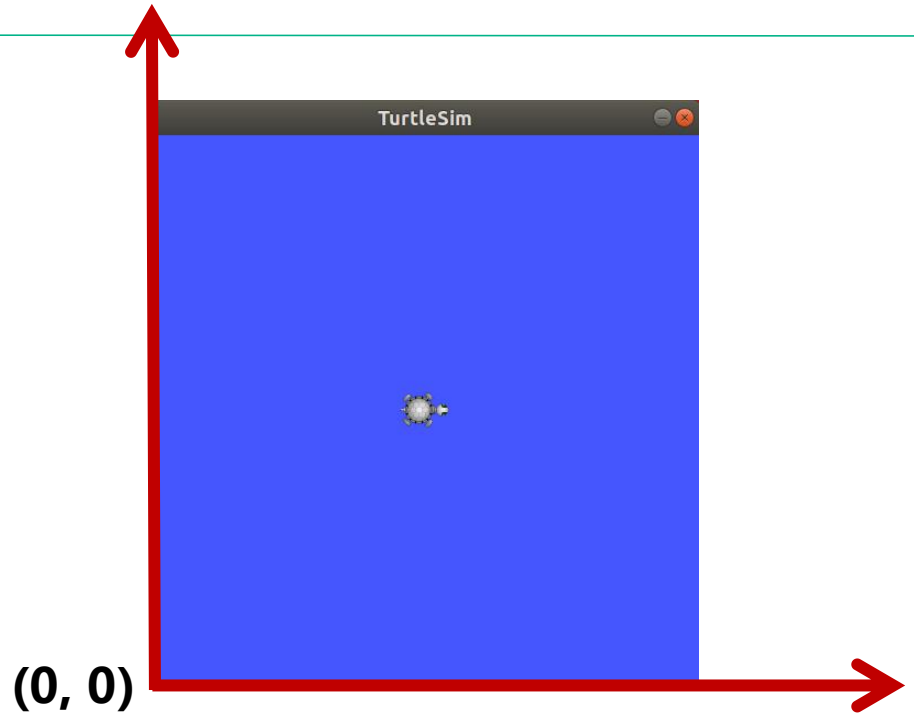
**2. 查看乌龟位姿信息的消息类型**

```
ros@ROS-VirtualBox:~$ rostopic info /turtle1/pose
Type: turtlesim/Pose

Publishers:
 * /turtlesim (http://ROS-VirtualBox:38661/)

Subscribers: None
```

```
ros@ROS-VirtualBox:~$ rosmsg info turtlesim/Pose
float32 x
float32 y
float32 theta
float32 linear_velocity
float32 angular_velocity
```




(1) 当前窗口可以看作是一个世界坐标系

(2) 两只乌龟可以看作点/局部坐标系

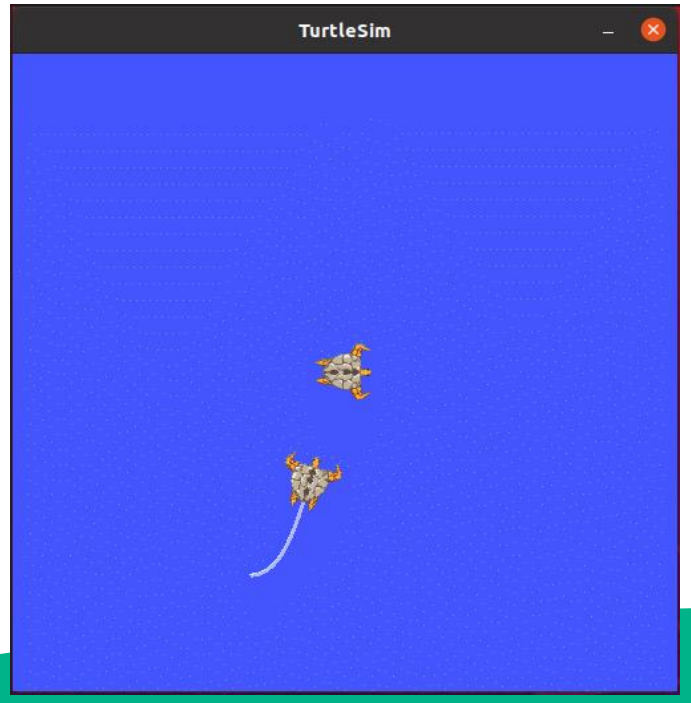
(3) 两只乌龟的位姿信息，可看作其在当前窗口所表示的世界坐标系内的点的坐标信息

## 1.1.1 Overview

3. 订阅两只乌龟的位姿信息，获取两只乌龟在世界坐标系的位置

4. 进行坐标变换，确定两只乌龟的相对位置关系

5. 根据相对位置关系生成速度信息，赋值给乌龟2，使得乌龟2靠近乌龟1




➢ 几点说明：

- 这一过程通过 tf 坐标变换来实现
- 一个 tf 系统由 tf 广播者(tf broadcaster)和 tf 监听者(tf listener)两个部分构成
- 乌龟1和乌龟2是两个 tf 广播者，它们不断的向外发送自己相对于世界坐标系的位置
- 另外有一个监听者负责维护两个乌龟的坐标系，通过 tf 的API系统计算两只乌龟的相对位置，并控制乌龟2跟随乌龟1
- 上述过程涉及到位姿信息的实时动态发布，因此需要有时间信息

## 1.1.2 Euler angles and quaternions

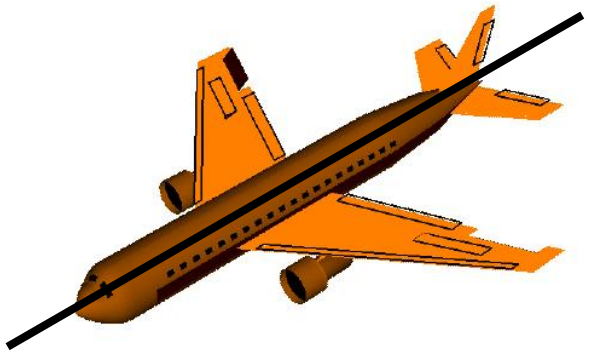
- 刚体的任一姿态(任一动坐标系)可以经由三次绕 X、Y、Z 坐标系的基本旋转得到，用三个角来描述就是欧拉角，分别为：
  - 翻滚(Roll) ----X
  - 俯仰(Pitch) --- Y
  - 偏航(Yaw) ----Z



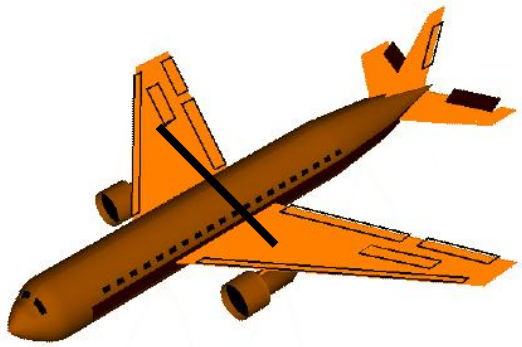
# 1.1.2 Euler angles and quaternions

- 翻滚(Roll)、俯仰(Pitch)与偏航(Yaw)

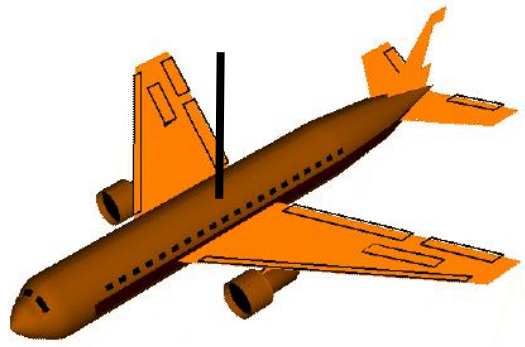
翻滚(Roll)



俯仰(Pitch)

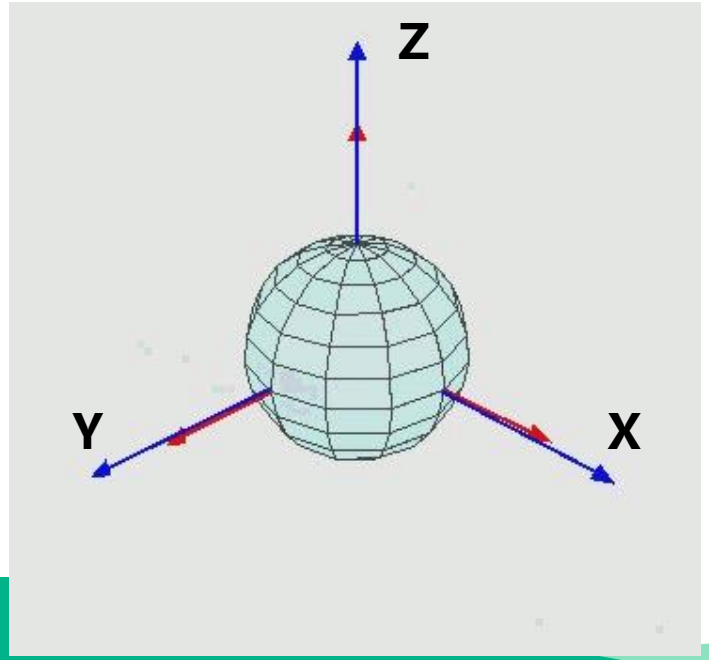


偏航(Yaw)



# 1.1.2 Euler angles and quaternions

- 欧拉角可以细分为欧拉角(Euler-angles)和泰特布莱恩角(Tait-Bryan-angles)。这两种方法都利用了笛卡尔坐标系的三轴作为旋转轴，主要区别在于旋转顺序：
  - 欧拉角的旋转顺序：绕 a 轴旋转某角度后，绕新生成的 b 轴旋转一个角度，最后绕两次旋转以后的 a 轴再旋转一个角度，以此表示最终的方向



# 1.1.2 Euler angles and quaternions

- **泰特布莱恩角的旋转顺序**：遍历笛卡尔坐标系的三轴，即前述的 Roll-Pitch-Yaw 角就是其中的情况




- 这两种方法都是在空间中用**最直观的方式和最少的参数**表示任意方向的通用方法，用它们表示方向**没有计算要求和容量需求的区别**

## 1.1.2 Euler angles and quaternions

- 欧拉角存在问题
  - (1) 不唯一性
  - (2) 万向节死锁
  - (3) 对复杂多次变换的情景，欧拉角的计算量会增大

### (1) 不唯一性

- **三维空间中的任意一个方向都可以通过至少两种不同欧拉角表示**，即当给定了目标角以后，可以很容易找到欧拉角表述的方向；但当我们获得了一个方向以后，却不一定能反推回目标欧拉角

参考资料：https://www.zhihu.com/question/47736315/answer/236808639

# 1.1.2 Euler angles and quaternions

### (2) 万向节死锁

- 假设我们要把 3D 物体旋转到某个想要的方向时，我们按照先以 x 轴为旋转轴旋转 α 角，然后绕 y 轴旋转 β 角，最后绕 z 轴旋转 γ 角这样的顺序实现（即按欧拉角来表示旋转）。上述过程可用旋转矩阵表示为：

$$R = \begin{bmatrix} \cos(\beta)\cos(\gamma) & \sin(\alpha)\sin(\beta)\cos(\gamma) - \sin(\gamma)\cos(\alpha) & \sin(\alpha)\sin(\gamma) + \sin(\beta)\cos(\alpha)\cos(\gamma) \\ \sin(\gamma)\cos(\beta) & \sin(\alpha)\sin(\beta)\sin(\gamma) + \cos(\alpha)\cos(\gamma) & -\sin(\alpha)\cos(\gamma) + \sin(\beta)\sin(\gamma)\cos(\alpha) \\ -\sin(\beta) & \sin(\alpha)\cos(\beta) & \cos(\alpha)\cos(\beta) \end{bmatrix}$$

# 1.1.2 Euler angles and quaternions

## (2) 万向节死锁

当我们的物体的初始方向就是已经绕 y 轴旋转了负 90 度，即 β 角取 $-\pi/2$ ，那么上面的旋转矩阵将会变成：

$$R = \begin{bmatrix} 0 & -\sin(\alpha+\gamma) & -\cos(\alpha+\gamma) \\ 0 & \cos(\alpha+\gamma) & -\sin(\alpha+\gamma) \\ 1 & 0 & 0 \end{bmatrix}$$

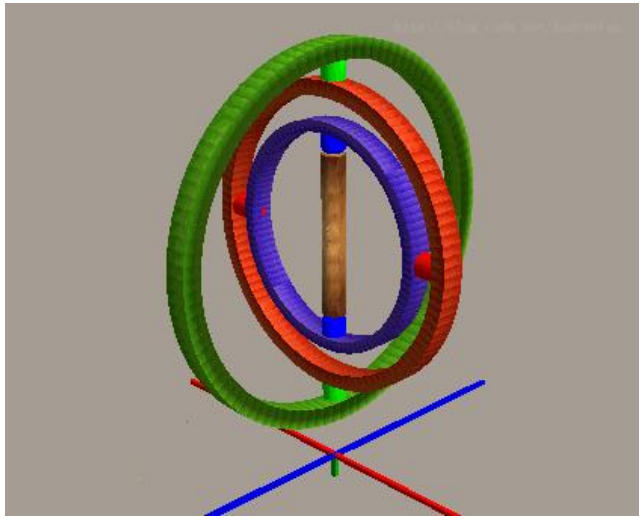
- 注意到 α 角和 γ 角在矩阵元素里是成对出现的，也就是说改变 α 角和改变 γ 角对旋转矩阵的影响是一样的
- 换句话说，原本有 3 个自由度，现在变成了 2 个自由度，这就是所谓的万向节死锁

# 1.1.2 Euler angles and quaternions

## (2) 万向节死锁



发生死锁



平衡环架(陀螺仪)：由彼此垂直的枢纽轴所组成的一组三只平衡环架，可使最内环架的物体维持旋转轴不变

“万向节死锁”原因，是由于欧拉角多对一的映射，造成一些旋转自由度的缺失，产生“死锁”

# 1.1.2 Euler angles and quaternions

## (3) 对复杂多次变换的情景，欧拉角的计算量会增大

- 为了避免欧拉角不唯一性，通常采用轴角表示的方式描述物体旋转
  - 轴角表示(Axis-Angle-Representation)：找到一根旋转轴，通过绕这根旋转轴旋转一次就可以得到目标方向
  - 三维空间中，任意旋转轴是唯一的，绕一个旋转轴旋转不同角度，对应的方向也是不同的。三维空间中任一位置的轴角表示是唯一的，不存在不唯一性问题
- 旋转轴通过向量 $[x, y, z]^T$ 来表示，旋转角度以通过一个度数 θ 来表示，轴角表示为：

$$(axis, angle) = \left( \begin{bmatrix} x \\ y \\ z \end{bmatrix}, \theta \right)$$

# 1.1.2 Euler angles and quaternions

- 四元数的引入
  - 引入四元数是为了更方便地计算轴角表示的方向变换
  - 万向节死锁的根源在于欧拉角的定义方式，万向节死锁的结果会导致丧失自由度，要规避万向节死锁，需要选择合适的旋转顺序。编程中很难规避死锁问题，所以很多时候都使用四元数实现三维物体旋转
  - 在大量变换的运算中，四元数的应用可以减少计算量和计算时存储占用的空间(解决欧拉角计算量大的问题)

# 1.1.2 Euler angles and quaternions

- 四元数的定义

  - 轴角表示(Axis-Angle-Representation)：$(axis, angle) = \left( \begin{bmatrix} x \\ y \\ z \end{bmatrix}, \theta \right)$

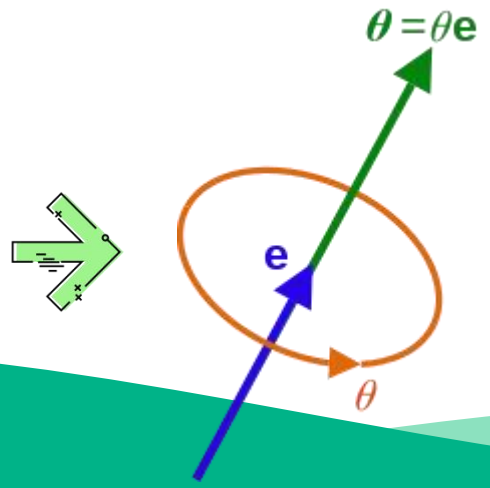
  - 四元数是一个超复数，其定义为：$q = w + ix + jy + kz$

    其中 i, j, k 是虚数单位，满足：$i^2 = j^2 = k^2 = -1$ ，且 $i \cdot j = k$ ， $j \cdot i = -k$

    w, x, y, z 是实数

  - 四元数用于计算轴角表示运算时，我们通常写成向量形式(vector representation)

$$q = [w, \vec{v}] = \left[ w, \begin{pmatrix} x \\ y \\ z \end{pmatrix} \right]$$



# 1.1.2 Euler angles and quaternions

- 四元数的使用
  - 四元数和欧拉角度可以互相转换
  - 给定一个欧拉旋转(X, Y, Z)（即分别绕x轴、y轴和z轴旋转X、Y、Z度），则对应的四元数为：

    x = sin(Y/2)sin(Z/2)cos(X/2)+cos(Y/2)cos(Z/2)sin(X/2)

    y = sin(Y/2)cos(Z/2)cos(X/2)+cos(Y/2)sin(Z/2)sin(X/2)

    z = cos(Y/2)sin(Z/2)cos(X/2)-sin(Y/2)cos(Z/2)sin(X/2)

    w = cos(Y/2)cos(Z/2)cos(X/2)-sin(Y/2)sin(Z/2)sin(X/2)

    q = ((x, y, z), w)

# 1.1.2 Euler angles and quaternions

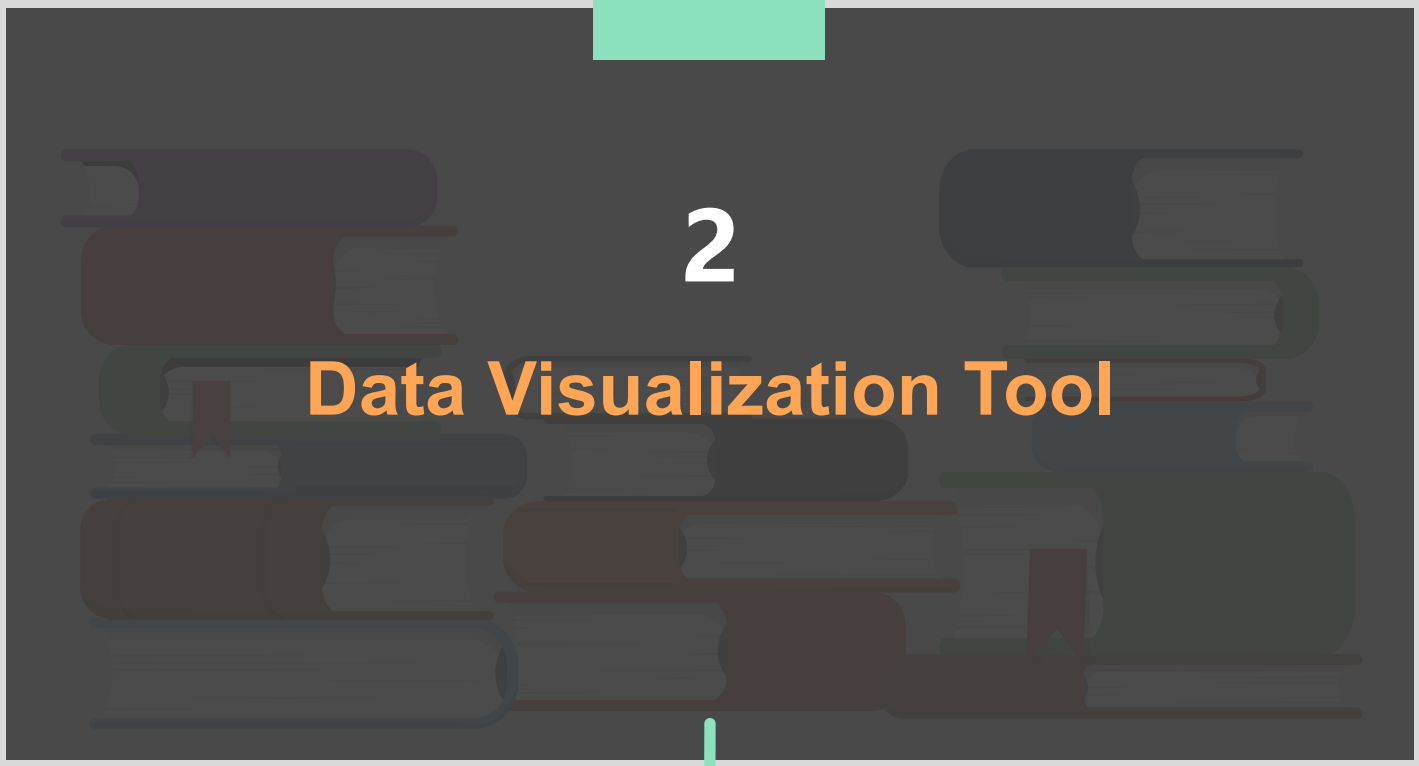
- 四元数的使用
  - ROS中，将 RPY坐标系下的 角度 转换为 四元数

```
#include “tf2/LinearMath/Quaternion.h”

tf2::Quaternion myQuaternion;
myQuaternion.setRPY(1.5707, 0, -1.5707);  // Create this quaternion from roll/pitch/yaw (in radians)
```

void setRPY (const tf2Scalar &roll, const tf2Scalar &pitch, const tf2Scalar &yaw)

Set the quaternion using fixed axis RPY.



# 2

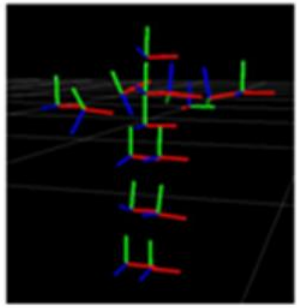
# Data Visualization Tool

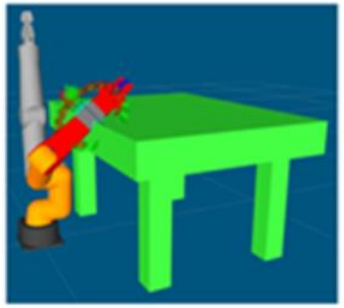
# 2.1 Rviz

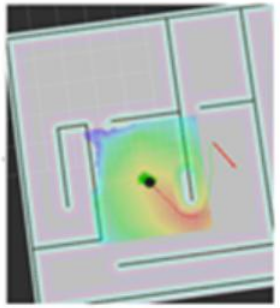
## 2.1.1 Overview

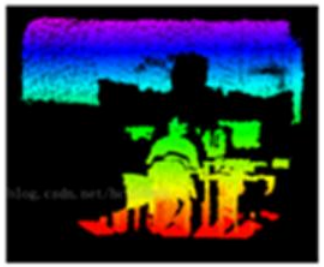
- 机器人系统中存在大量数据，这些数据在计算过程中往往都处于数据形态
- 数据形态的值不利于开发者去感受数据所描述的内容，需要将数据可视化显示
- 举例：机器人模型的可视化、图像数据的可视化、地图数据的可视化等


机器人模型


坐标


运动规划


导航


点云


图像


SLAM

# 2.1 Rviz

## 2.1.1 Overview

- Rviz是一款三维可视化工具，很好的兼容了各种基于 ROS 软件框架的机器人平台
- 在 Rviz 中，可以使用 XML 对机器人、周围物体等任何实物进行尺寸、质量、位置、材质、关节等属性的描述，并且在界面中呈现出来
- Rviz 可以通过图形化的方式，实时显示机器人传感器的信息、机器人的运动状态、周围环境的变化等

- Rviz 安装：

```
$ sudo apt install ros-noetic-rviz
```

- Rviz 运行：

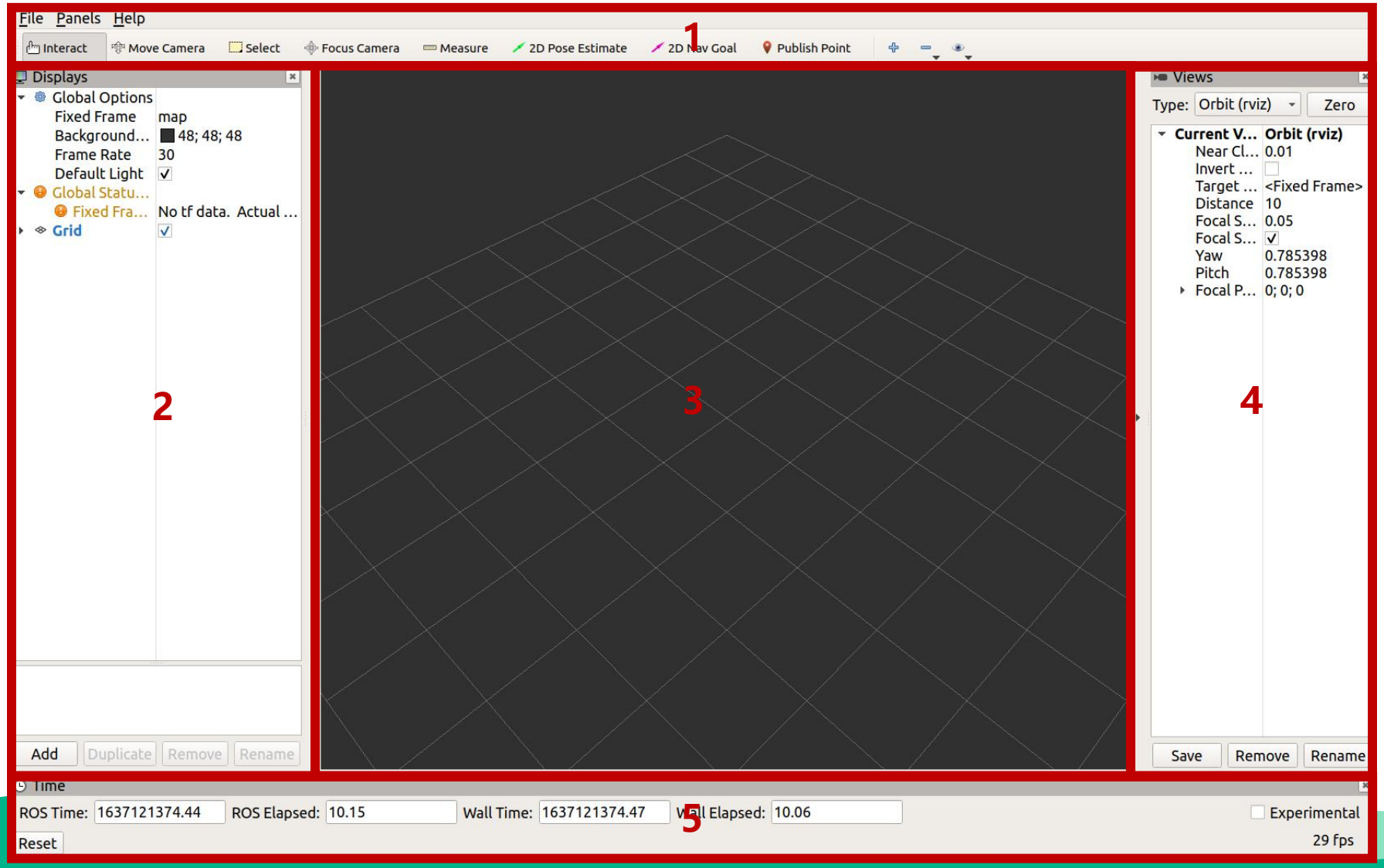
```
$ roscore
```

```
$ rosrun rviz rviz
```

```
$ rviz
```

# 2.1 Rviz

## 2.1.1 Overview




1、**工具栏**，提供视角控制、目标设置、发布地点等工具

2、**显示项列表**，用于显示当前选择的插件，可以配置每个插件的属性

3、**3D视图区**，用于可视化显示数据，目前没有任何数据，所以显示黑色

4、**视角设置区**，可以选择多种观测视角

5、**时间显示区**，显示当前的系统时间和ROS时间

# 2.1 Rviz

## 2.1.2 Data visualization： http://wiki.ros.org/rviz/UserGuide

1、假设需要可视化的数据以对应的消息类型发布，在Rviz中使用、插件订阅该消息即可实现显示

2、Rviz默认可以显示下表所示的通用类型数据，其中包含坐标轴、摄像头图像、地图、激光等数据

| Name | Description | Messages Used |
|---|---|---|
| Axes | Displays a set of Axes | |
| Effort | Shows the effort being put into each revolute joint of a robot. | sensor_msgs/JointStates |
| Camera | Creates a new rendering window from the perspective of a camera, and overlays the image on top of it. | sensor_msgs/Image, sensor_msgs/CameraInfo |
| Grid | Displays a 2D or 3D grid along a plane | |
| Grid Cells | Draws cells from a grid, usually obstacles from a costmap from the navigation stack. | nav_msgs/GridCells |
| Image | Creates a new rendering window with an Image. Unlike the Camera display, this display does not use a CameraInfo. *Version: Diamondback+* | sensor_msgs/Image |
| InteractiveMarker | Displays 3D objects from one or multiple Interactive Marker servers and allows mouse interaction with them. *Version: Electric+* | visualization_msgs/InteractiveMarker |
| Laser Scan | Shows data from a laser scan, with different options for rendering modes, accumulation, etc. | sensor_msgs/LaserScan |
| Map | Displays a map on the ground plane. | nav_msgs/OccupancyGrid |
| Markers | Allows programmers to display arbitrary primitive shapes through a topic | visualization_msgs/Marker, visualization_msgs/MarkerArray |
| Path | Shows a path from the navigation stack. | nav_msgs/Path |
| Point | Draws a point as a small sphere. | geometry_msgs/PointStamped |
| Pose | Draws a pose as either an arrow or axes. | geometry_msgs/PoseStamped |
| Pose Array | Draws a "cloud" of arrows, one for each pose in a pose array | geometry_msgs/PoseArray |
| Point Cloud(2) | Shows data from a point cloud, with different options for rendering modes, accumulation, etc. | sensor_msgs/PointCloud, sensor_msgs/PointCloud2 |
| Polygon | Draws the outline of a polygon as lines. | geometry_msgs/Polygon |
| Odometry | Accumulates odometry poses from over time. | nav_msgs/Odometry |
| Range | Displays cones representing range measurements from sonar or IR range sensors. *Version: Electric+* | sensor_msgs/Range |
| RobotModel | Shows a visual representation of a robot in the correct pose (as defined by the current TF transforms). | |
| TF | Displays the tf transform hierarchy. | |
| Wrench | Draws a wrench as arrow (force) and arrow + circle (torque) | geometry_msgs/WrenchStamped |
| Oculus | Renders the RViz scene to an Oculus headset | |

# 2.1 Rviz

## 2.1.2 Data visualization

| 插件名 | 描述 | 消息类型 |
| --- | --- | --- |
| Axes | 显示坐标轴 | - |
| Effort | 显示机器人转动关节的力 | sensor_msgs/JointStates |
| Camera | 打开一个新窗口并显示摄像头图像 | sensor_msgs/Image<br>sensor_msgs/CameraInfo |
| Grid | 显示 2D 或者 3D 栅格 | |
| Grid Cells | 显示导航功能包中代价地图的障碍物栅格信息 | nav_msgs/GridCells |
| Image | 打开一个新窗口并显示图像信息<br>（不需要订阅摄像头校准信息） | sensor_msgs/Image |
| InteractiveMarker | 显示 3D 交互式标记 | visualization_msgs/InteractiveMarker |
| Laser Scan | 显示激光雷达数据 | sensor_msgs/LaserScan |
| Map | 在大地平面上显示地图信息 | nav_msgs/OccupancyGrid |
| Markers | 绘制各种基本形状（箭头、立方体、球体、圆柱体、线带、线列表、立方体列表、球体列表、点、文本、mesh 数据、三角形列表等） | visualization_msgs/Marker<br>visualization_msgs/MarkerArray |
| Path | 显示导航过程中的路径信息 | nav_msgs/Path |

# 2.1 Rviz

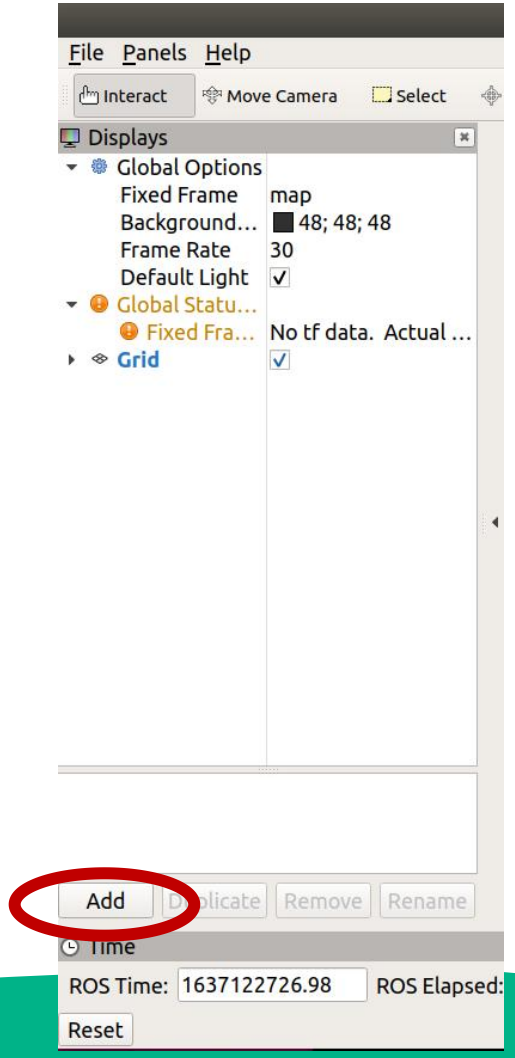
## 2.1.2 Data visualization

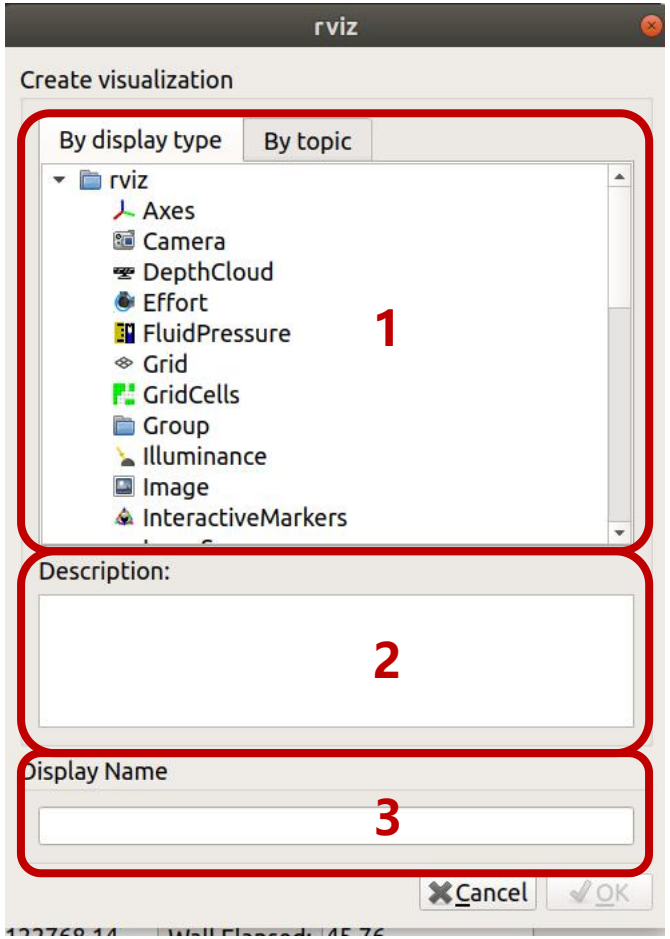
| | | |
|---|---|---|
| Point | 使用圆球体绘制一个点 | geometry_msgs/PointStamped |
| Pose | 使用箭头或者坐标轴的方式绘制一个位姿 | geometry_msgs/PoseStamped |
| Pose Array | 根据位姿列表，绘制一组位姿箭头 | geometry_msgs/PoseArray |
| Point Cloud(2) | 显示点云数据 | sensor_msgs/PointCloud<br>sensor_msgs/PointCloud2 |
| Polygon | 绘制多边形轮廓 | geometry_msgs/Polygon |
| Odometry | 绘制一段时间内的里程计位姿信息 | nav_msgs/Odometry |
| Range | 显示声呐或者红外传感器反馈的测量数据(锥形范围) | sensor_msgs/Range |
| RobotModel | 显示机器人模型（根据 TF 变换确定机器人模型的位姿） | - |
| TF | 显示 TF 变换的层次关系 | - |
| Wrench | 显示力信息（力用箭头表示，转矩用箭头和圆表示） | geometry_msgs/WrenchStamped |

# 2.1 Rviz

## 2.1.2 Data visualization

- 如何操作?




1、插件列表

2、插件描述

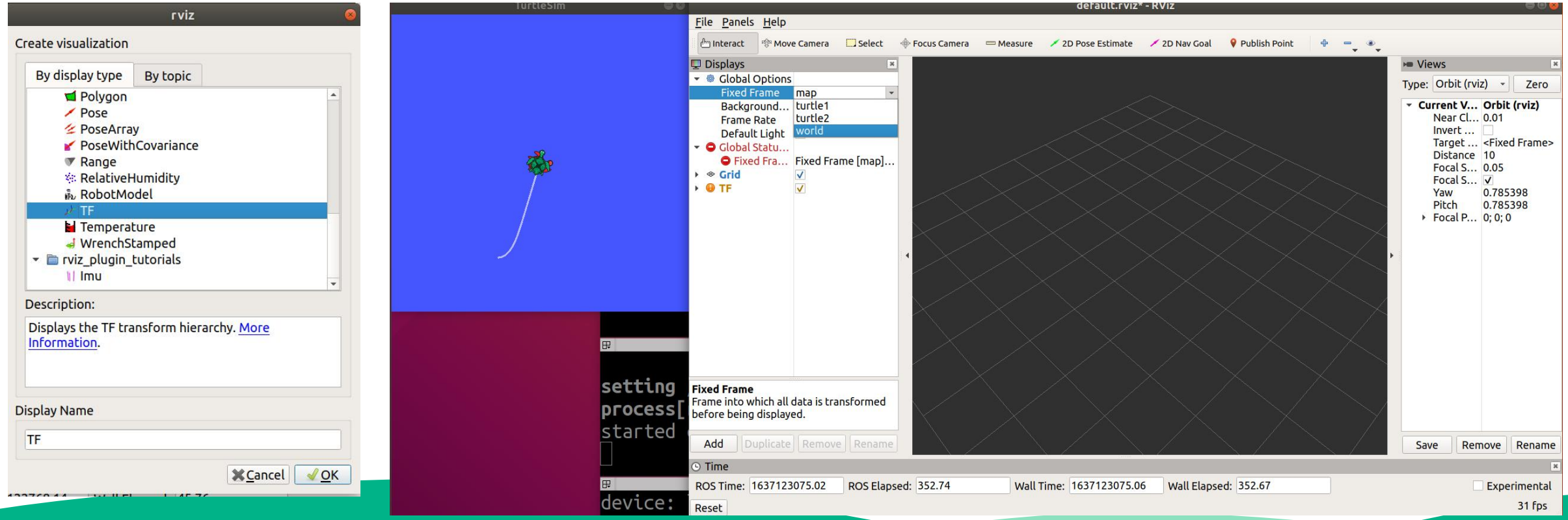
3、插件名称

# 2.1 Rviz

## 2.1.2 Data visualization

- **TF操作演示：乌龟跟随**

```
$ roslaunch turtle_tf2 turtle_tf2_demo.launch
```



# 2.1 Rviz

## 2.1.2 Data visualization

- **TF操作演示：乌龟跟随**

```
Use arrow keys to move the turtle. 'q' to quit.
process[turtle_pointer-5]: started with pid [5323]
/usr/bin/env: “python”: 没有那个文件或目录
[turtle1_tf2_broadcaster-3] process has died [pid 5315, exit code 127, cmd /opt/ros/noetic/lib/turtle_tf2/turtle_tf2
_broadcaster.py __name:=turtle1_tf2_broadcaster __log:=/home/qlf/.ros/log/7bc2cc30-8f81-11ee-a033-371c9d008e18/turtl
e1_tf2_broadcaster-3.log].
log file: /home/qlf/.ros/log/7bc2cc30-8f81-11ee-a033-371c9d008e18/turtle1_tf2_broadcaster-3*.log
[turtle2_tf2_broadcaster-4] process has died [pid 5316, exit code 127, cmd /opt/ros/noetic/lib/turtle_tf2/turtle_tf2
_broadcaster.py __name:=turtle2_tf2_broadcaster __log:=/home/qlf/.ros/log/7bc2cc30-8f81-11ee-a033-371c9d008e18/turtl
e2_tf2_broadcaster-4.log].
log file: /home/qlf/.ros/log/7bc2cc30-8f81-11ee-a033-371c9d008e18/turtle2_tf2_broadcaster-4*.log
[turtle_pointer-5] process has died [pid 5323, exit code 127, cmd /opt/ros/noetic/lib/turtle_tf2/turtle_tf2_listener
.py __name:=turtle_pointer __log:=/home/qlf/.ros/log/7bc2cc30-8f81-11ee-a033-371c9d008e18/turtle_pointer-5.log].
log file: /home/qlf/.ros/log/7bc2cc30-8f81-11ee-a033-371c9d008e18/turtle_pointer-5*.log
```

```
$ sudo rm -f /usr/bin/python
```

```
$ whereis python
```

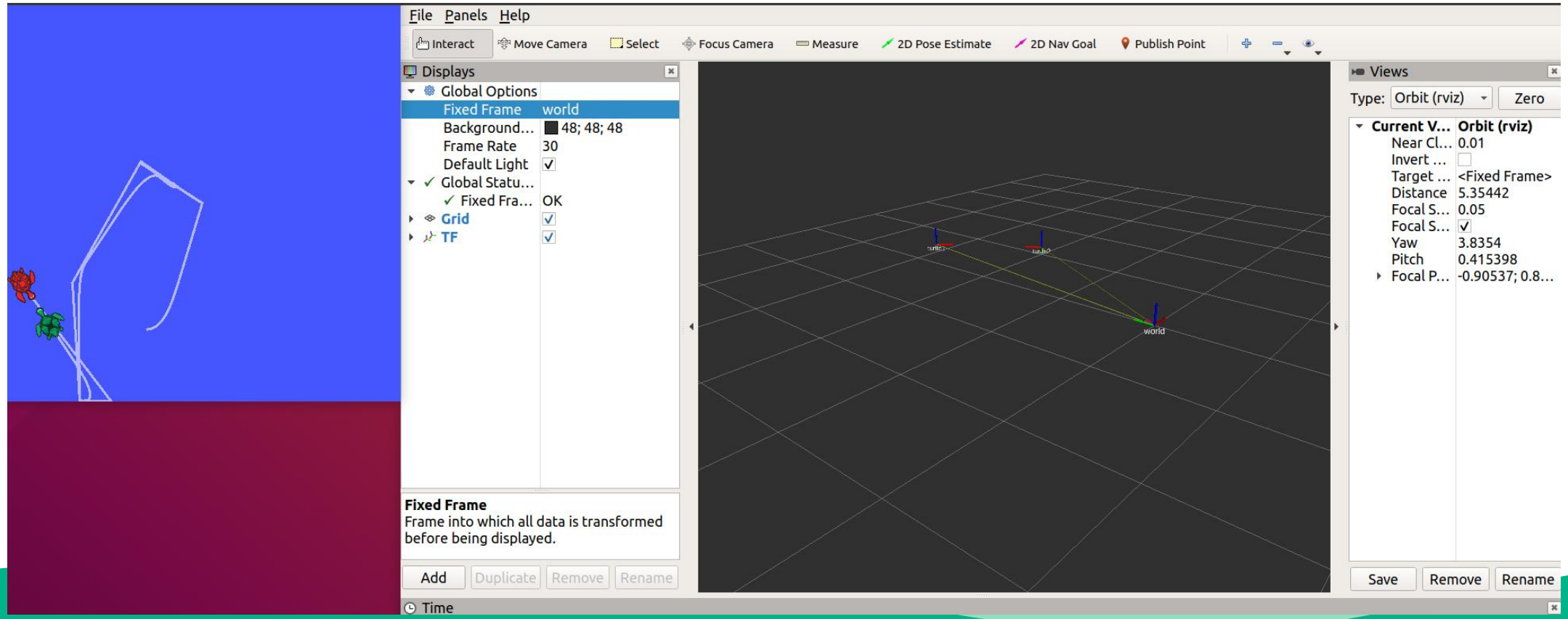
```
$ sudo ln -s /usr/bin/python3.8 /usr/bin/python
```

# 2.1 Rviz

## 2.1.2 Data visualization

- **TF操作演示：乌龟跟随**

```
$ rviz
```



# Summary

1. TF2

   a) 概念、作用、

   b) 分析乌龟跟随案例的实现流程

   c) 欧拉角和四元数

      1. 欧拉角定义、存在问题

      2. 轴角表示

      3. 四元数引入、定义和使用

2. Rviz

   a) Rviz作用

   b) 基本使用方案

   c) 乌龟跟随案例演示

# Q&A

今日事今日毕

Question?